

Covariate Shift Regularization with Kernels

Suvinava Basak

February 13, 2025

Abstract

Covariate shift, a fundamental challenge in supervised learning, occurs when the distribution of input features differs between training and test datasets while the conditional distribution of the target given the input remains unchanged. This work explores covariate shift through the lens of kernel-based learning, particularly in linear regression settings. We analyze theoretical foundations, propose strategies for mitigating covariate shift using importance weighting with weights estimated by kernel density estimation and self-attention mechanisms.

We conduct experiments using synthetic data to assess the effect of mean shift on model performance, comparing ordinary least squares and gradient descent estimators. Our findings highlight the importance of robust learning strategies in the presence of covariate shift and emphasize the role of kernel-based techniques in domain adaptation and distributionally robust optimization.

Contents

1	Introduction	6
2	Covariate Shift	7
2.1	Covariate Shift in Kernel Methods	8
3	Strategies for Covariate Shift	11
3.1	Importance Weighting	11
3.1.1	Kernel Density Estimation	12
3.1.2	Self-Attention Mechanism	13
3.1.3	Kernel Mean Matching (KMM)	16
3.1.4	Logistic Regression-based Density Ratio Estimation	16
3.1.5	Kullback-Leibler Importance Estimation Procedure (KLIEP) . . .	16
3.1.6	Least-Squares Importance Fitting (LSIF)	16
3.2	Kernel Ridge Regression under Covariate Shift	17
3.3	Alternative Strategies	18
3.3.1	Domain Adaptation	18
3.3.2	Transfer Learning	18
3.3.3	Robust Risk Minimization in RKHS	18
4	Supervised Learning under Covariate Shift	19
4.1	Goal of Supervised Learning Algorithms	19
4.2	Introducing Covariate Shift to the Model	20
4.3	Empirical Risk Minimization and its Importance Weighted Variants . . .	21
4.3.1	Empirical Risk Minimization	21
4.3.2	Importance Weighted Variant of ERM	22
4.3.3	Other Variants of Importance Weighted ERM	23
5	State of the Art in Covariate Shift Regularization	24
5.1	When Is Importance Weighting Necessary?	24
5.2	Optimal Learning Under Covariate Shift	25
5.3	RKHS-based Theoretical Analysis and Generalization Bounds	25
5.4	Covariate Shift Adaptation via Importance-Weighted Cross-Validation . .	25
5.5	Covariate Shift Adaptation in Neural Networks and Decision Trees	26
5.5.1	Neural Networks and Domain Adaptation	26
5.5.2	Decision Trees and Reweighting Methods	26
6	Experiment Setup	28
6.1	Covariate Shift in Linear Regression	28
6.2	Effect of Mean Shift on MSE	29

6.2.1	Ordinary Least Square (OLS) Estimator	29
6.2.2	Gradient Descent Estimator	30
6.2.3	Comparison between OLS and GD estimator (Effect of Mean Shift)	34
6.3	Regularizing Covariate Shift	36
6.3.1	Importance Weighting using Kernel Density Estimation	37
6.3.2	Self-attention Mechanism	42
7	Conclusion	45
	Bibliography	46

1 Introduction

The goal of any supervised learning problem is to develop a model to infer an unknown relationship between the input data (features) and output data (labels for classification or outcome for regression) from the training samples. This predicted relationship, then, can be used to estimate the output values (labels or outcome) for unseen test input data points. Traditionally, while building a supervised learning model, we commonly assume that the training and test data are drawn from the same distribution, i.e., the input points in the training set and the input points used for testing follow the same probability distribution [Bishop, 1995; Duda et al., 2001; Vapnik, 1998; Hastie et al., 2009].

However, in real-world applications, this assumption is frequently violated due to multiple reasons. When this assumption is violated, standard empirical risk minimization (ERM) can lead to models that fail to generalize poorly for the new, unseen data from the test distributions. This phenomenon is known as *covariate shift*, where the marginal distribution of the input features differs between training and test data while the conditional distribution of the output given the input remains unchanged [Shimodaira, 2000]. This shift often occurs due to biases in data collection, domain variations, or changes in the underlying data-generating process [Sugiyama et al., 2000]. This is very common in real-world applications such as medical imaging, finance, and NLP (Natural Language Processing), where the data acquisition process introduces sampling bias.

Covariate shift poses a significant challenge in supervised learning, as it can significantly degrade the performance of the machine learning models. It arises in various applications, including off-policy reinforcement learning [Shelton, 2001], spam filtering [Bickel and Scheffer, 2007], bioinformatics [Baldi et al., 1998; Borgwardt et al., 2006], and brain-computer interfacing [Wolpaw et al., 2002]. Additionally, in economics, sample selection bias [Heckman, 1979] can also manifest as a form of covariate shift.

The goal of this project is to explore the problem of covariate shift in supervised learning through the lens of kernel methods, particularly in the linear regression settings. We examine theoretical foundations, key methodologies in mitigating covariate shift using kernel-based learning techniques in supervised learning context.

2 Covariate Shift

Formally, let $\mathcal{X} \in \mathbb{R}^d$ be the input space from which samples x are drawn and $\mathcal{Y} \in \mathbb{R}$ (or $\mathcal{Y} \in \{0, 1\}$ in classification tasks) be the output space. In the standard supervised learning setting, we have access to a training dataset $\{(x_i, y_i)\}_{i=1}^n$ samples from the joint distribution $\mathbb{P}_{\text{train}}(X, Y)$ and we aim to learn a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ that generalizes well to the new, unseen data from the test distribution $\mathbb{P}_{\text{test}}(X, Y)$. We do that by selecting the function $f : \mathcal{X} \rightarrow \mathcal{Y}$ that minimizes the *expected risk*:

$$\mathcal{R}(f) = \mathbb{E}_{\mathbb{P}_{\text{train}}(X, Y)}[\ell(f(X; \theta), Y)]$$

where, $\ell : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ is a suitable loss function and θ is the parameter that we aim to learn.

Covariate shift occurs when (1) the marginal distribution of the input features (covariates) X differs between the training and testing phases, while (2) the conditional distribution of the output given the input $\mathbb{P}(Y | X)$ remains unchanged. This means that while the relationship between the input features and the target variable doesn't change, the way the input features are distributed does. Mathematically, we can break it down as below.

Let us define the training and test distributions by $\mathbb{P}_{\text{train}}(X, Y)$ and $\mathbb{P}_{\text{test}}(X, Y)$ respectively. Then under covariate shift, it holds that:

- **Input Distribution Change:** The probability distribution $\mathbb{P}(X)$ of the input variables X changes between training and testing.

$$\mathbb{P}_{\text{train}}(X) \neq \mathbb{P}_{\text{test}}(X)$$

- **Conditional Distribution Stability:** The conditional distribution $\mathbb{P}(Y | X)$ of the target variable Y given the input variables X remains the same.

$$\mathbb{P}_{\text{train}}(Y | X) = \mathbb{P}_{\text{test}}(Y | X)$$

This mismatch between $\mathbb{P}_{\text{train}}(X)$ and $\mathbb{P}_{\text{test}}(X)$ can degrade the model performance to a great extent, as standard empirical risk minimization (ERM) minimizes the risk under $\mathbb{P}_{\text{train}}(X)$ rather than $\mathbb{P}_{\text{test}}(X)$. In other words, the model was trained on a different distribution of input features. The model may not generalize well if the training data does not adequately represent the test data. Our goal is to adjust for this change while learning the function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ during model training based on the data $(x_i, y_i)_{i=1}^n \sim \mathbb{P}_{\text{train}}(X, Y)$.

2.1 Covariate Shift in Kernel Methods

Kernel methods operate by embedding input data $X \in \mathbb{R}^d$ into a high-dimensional feature space $\mathcal{H}$ through a mapping function $\phi : \mathbb{R}^d \rightarrow \mathcal{H}$, where $\mathcal{H}$ is a reproducing kernel Hilbert space (RKHS) with inner product $\langle \cdot, \cdot \rangle_{\mathcal{H}}$. Algorithms such as kernel ridge regression (KRR), support vector machines (SVMs), and Gaussian processes (GPs) operate in such RKHS [Schölkopf and Smola, 2002]. Instead of explicitly computing the mapping $\phi(X)$, kernel methods compute inner products in the feature space using a **kernel function** $k : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$:

$$k(X_i, X_j) = \langle \phi(X_i), \phi(X_j) \rangle_{\mathcal{H}}$$

Note: A Reproducing Kernel Hilbert Space (RKHS) is a special type of Hilbert space that is associated with a kernel function and has reproducing property. For any function f in RKHS and any point x , we have:

$$f(x) = \langle f, K(x, \cdot) \rangle_{\mathcal{H}}$$

This means that evaluating $f(x)$ is equivalent to taking an inner product with the kernel function.

Common kernel functions include:

- **Linear kernel:** $k(X_i, X_j) = X_i^\top X_j$
- **Radial Basis Function (RBF) kernel:** $k(X_i, X_j) = \exp(-\gamma \|X_i - X_j\|^2)$
- **Polynomial kernel:** $k(X_i, X_j) = (X_i^\top X_j + c)^d$, where d is the degree of polynomial kernel

In the context of kernel methods, the problem of covariate shift is particularly relevant when learning from data mapped into a reproducing kernel Hilbert space (RKHS). Since kernel methods rely on similarity measures derived from the feature distribution, a shift in the marginal distribution can lead to biased estimates and deteriorated generalization performance. Given a kernel function $k(X_i, X_j)$, these methods rely on the assumption that training and test distributions are aligned in the RKHS.

When covariate shift occurs, kernel-based models face the following issues:

1. Biased Function Estimation

In the kernel framework, for many kernel-based learning methods, such as kernel least squares regression or SVMs, the objective is to find a function f in the reproducing Kernel Hilbert Space (RKHS) $\mathcal{H}$ induced by the kernel function $k(\cdot, \cdot)$. We do so by minimizing the empirical risk over the given training data $\{(X_i, Y_i)\}_{i=1}^n$:

$$\min_f \mathcal{R}(f) = \min_f \mathbb{E}_{\mathbb{P}_{\text{train}}}[\ell(Y, f(X))] \quad (2.1)$$

where, $\ell(\cdot, \cdot)$ is a loss function (e.g., squared error for regression, cross-entropy for classification). However, under covariate shift the risk we care about is with respect to the test distribution $\mathbb{P}_{\text{test}}$:

$$\min_f \mathcal{R}_{\mathbb{P}_{\text{test}}}(f) = \min_f \mathbb{E}_{\mathbb{P}_{\text{test}}}[\ell(Y, f(X))] \quad (2.2)$$

Now, since $\mathbb{P}_{\text{train}}(X) \neq \mathbb{P}_{\text{test}}(X)$, the empirical risk computed from training samples no longer represents the expected test risk, leading to biased estimators [Shimodaira, 2000].

2. Shifted Kernel Mean Embeddings

Kernel methods provide a powerful framework for nonparametric learning by embedding probability distributions into a reproducing kernel Hilbert space (RKHS) [Gretton et al., 2009]. The central idea is to map a probability distribution $\mathbb{P}$ to a feature space through its expectation:

$$\mu_{\mathbb{P}} = \mathbb{E}_{X \sim \mathbb{P}}[\phi(X)] \quad (2.3)$$

where $\phi(X)$ is a feature map induced by a positive definite kernel. Such feature space representations based on kernel mean embeddings allows us to compare distributions in RKHS using the Maximum Mean Discrepancy (MMD):

$$\text{MMD}(P, Q) = \|\mu_P - \mu_Q\|_{\mathcal{H}} \quad (2.4)$$

Under covariate shift, the training and test distributions differ in their marginal distribution of inputs, i.e., $\mathbb{P}_{\text{train}}(X) \neq \mathbb{P}_{\text{test}}(X)$, while keeping the conditional distribution of labels given inputs the same: $\mathbb{P}(Y | X)$ remains unchanged in both training and test set. This shift results in a discrepancy between the mean embeddings:

$$\mu_{\text{train}} = \mathbb{E}_{X \sim \mathbb{P}_{\text{train}}}[\phi(X)] \neq \mu_{\text{test}} = \mathbb{E}_{X \sim \mathbb{P}_{\text{test}}}[\phi(X)] \quad (2.5)$$

3. Kernel-based Generalization Bounds

Theoretical guarantees for generalization in RKHS often involve a complexity term (e.g. Rademacher complexity) dependent on the distribution of training samples. For models based on Reproducing Kernel Hilbert Spaces (RKHS), this complexity term depend on the spectral properties of kernel matrices formed using the training data. [Mohri et al., 2018]

When covariate shift occurs, i.e., when $\mathbb{P}_{\text{train}}(X) \neq \mathbb{P}_{\text{test}}(X)$ while $\mathbb{P}(Y | X)$ remains unchanged, the spectral properties of the kernel matrix change, affecting generalization bounds. This alteration affects Rademacher complexity, which quantify the capacity of a function class in terms of how well the learned function can fit random noise and generalize to unseen test data.

Given a function class $\mathcal{F}$ and training points $x_1, x_2, \dots, x_n \sim \mathbb{P}_{\text{train}}(X)$, the empirical Rademacher complexity is defined as:

$$\hat{\mathcal{R}}_n(\mathcal{F}) = \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i) \right] \quad (2.6)$$

where, σ_i are Rademacher variables (independent uniform random variables taking values ± 1). In the RKHS setting with a kernel k , a standard bound [Mohri et al., 2018] states:

$$\hat{\mathcal{R}}_n(\mathcal{F}) \leq \frac{B}{n} \sqrt{\sum_{i=1}^n k(X_i, X_i)} \quad (2.7)$$

where B is an upper bound on the norm of functions in the RKHS.

When training and test distributions differ, the kernel matrix K , whose entries are $K_{ij} = K(X_i, X_j)$, has different spectral properties under $\mathbb{P}_{\text{train}}$ and $\mathbb{P}_{\text{test}}$. Specifically, if $\lambda_1 \geq \lambda_2 \geq \dots$ are the eigenvalues of K under $\mathbb{P}_{\text{train}}$, these eigenvalues change when computed under $\mathbb{P}_{\text{test}}$. A heavier-tailed test distribution (e.g., more variance in feature space) can increase the effective dimension of the RKHS function class, increasing Rademacher complexity. A typical generalization bound for an RKHS-based learning function f trained under $\mathbb{P}_{\text{train}}$ but tested on $\mathbb{P}_{\text{test}}$ takes the form:

$$\mathbb{E}_{X \sim \mathbb{P}_{\text{test}}} [|f(X) - f^*(X)|] \leq \mathcal{O} \left(\sqrt{\frac{\hat{\mathcal{R}}_n^w(f)}{n}} + \text{MMD}(\mathbb{P}_{\text{train}}, \mathbb{P}_{\text{test}}) \right) \quad (2.8)$$

where, f^* is the optimal function under $\mathbb{P}_{\text{test}}$ and $\hat{\mathcal{R}}_n^w(f)$ is the weight-adjusted empirical Rademacher complexity. The MMD term highlights that *greater distribution shift increases generalization error*.

Such misalignment due to the distribution shift influences how well the learned function generalizes to test data. [Mohri et al., 2018] provides theoretical frameworks for different techniques such as importance weighting (e.g., Kernel Mean Matching (KMM)) or distributionally robust optimization, to compensate for such shifts and their effects. It can be shown that effective dimension growth under covariate shift leads to weaker generalization guarantees.

3 Strategies for Covariate Shift

There are several techniques to handle covariate shift by adjusting the distribution mismatch between train and test datasets. In this section, we will discuss some of the most prominent methods to handle covariate shift.

3.1 Importance Weighting

One of the most popular approaches for handling covariate shift is to introduce *importance weights* that adjusts the empirical risk to better reflect the test distribution [Shimodaira, 2000].

If $p_{\text{train}}(X)$ and $p_{\text{test}}(X)$ are the probability density functions corresponding to the input distribution $\mathbb{P}_{\text{train}}(X)$ (also called *source distribution*) and $\mathbb{P}_{\text{test}}(X)$ (also called *target distribution*) respectively, then the importance weights for individual samples x_i are given by:

$$\omega(x_i) = \frac{p_{\text{test}}(x_i)}{p_{\text{train}}(x_i)} = \frac{d\mathbb{P}_{\text{test}}(x_i)}{d\mathbb{P}_{\text{train}}(x_i)} \quad (3.1)$$

where $w : X \rightarrow \mathbb{R}_+$ is the *Radon-Nikodym derivative* of the target measure with respect to the source measure.

The standard *empirical risk* with a loss function $\ell : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ to learn the parameter θ [Vapnik, 1998; Schölkopf and Smola, 2002], as stated below:

$$\hat{\mathcal{R}}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{f}(x_i; \theta)) \quad (3.2)$$

now becomes *importance weighted empirical risk* as below:

$$\hat{\mathcal{R}}_w(\theta) = \frac{1}{n} \sum_{i=1}^n \omega(x_i) \ell(y_i, \hat{f}(x_i; \theta)) \quad (3.3)$$

This ensures that the training data is effectively "corrected" to resemble the test distribution. We have to note that in practical situations, we do not know the underlying probability distributions of both train and test data, so the individual importance weights are not available to us. Therefore, we need a strategy to estimate the probability density functions from the data to learn the weights.

3.1. Importance Weighting

3.1.1 Kernel Density Estimation

Given a dataset $X = \{x_1, x_2, \dots, x_n\}$, the estimated probability density function $\hat{p}(x)$ at any point x is computed as:

$$\hat{p}(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right) \quad (3.4)$$

where, n is the number of data points, h is the bandwidth parameter to control the smoothness of the density estimate, $K(\cdot)$ is the kernel function and x_i are the observed data points. For example, if we are using *Gaussian Kernel*:

$$K(u) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}u^2} \quad (3.5)$$

Thus the estimated density simplifies to:

$$\hat{p}(x) = \frac{1}{nh\sqrt{2\pi}} \sum_{i=1}^n e^{-\frac{(x-x_i)^2}{2h^2}} \quad (3.6)$$

We also note that bandwidth (h) controls the smoothness of the kernel density estimate. A smaller bandwidth produces a more detailed (possibly noisy) density estimate while a larger bandwidth produces a smoother density estimate, which might oversmooth the data and lose details.

Once we learn the density function $p_{\text{source}}(x)$ and $p_{\text{target}}(x)$, we take the log-likelihood of the training data points under both $p_{\text{source}}(x)$ and $p_{\text{target}}(x)$.

The rationale behind taking the log-likelihood is that KDE estimates the probability density function $p(x)$, which is typically a very small number, especially in high-dimensional spaces. Since probabilities are bounded between 0 and 1, direct computation of density ratios according to 3.1 can involve extremely small values, leading to numerical underflow (loss of precision in floating-point calculations) in most programming languages like Python. Instead, we compute $\log p(x)$ which avoids dealing with very small numbers and keeps computations stable. So now, instead of 3.1 the importance weight ratios become:

$$\log \omega(x) = \log p_{\text{test}}(x) - \log p_{\text{train}}(x) \quad (3.7)$$

Let $M_\omega : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be the diagonal matrix of importance weights diagonal entries $(\omega(x_1), \dots, \omega(x_n))$ for n data points in the sample, i.e.,

$$M_\omega = \begin{bmatrix} \omega(x_1) & 0 & \cdots & 0 \\ 0 & \omega(x_2) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \omega(x_n) \end{bmatrix} \quad (3.8)$$

Ordinary Least Squares (OLS)

As explained in section 3.1 equation 3.3, the *importance weighted ordinary least square estimator* can be obtained by minimizing the empirical risk on training data with the additional effect of the importance weights:

$$\hat{\theta}_{\text{IWOLS}} = \arg \min_{\theta \in \mathbb{R}^d} \left[\frac{1}{n} \sum_{i=1}^n \omega(x_i) \left(\langle \hat{\theta}, x_i \rangle - y_i \right)^2 \right] \quad (3.9)$$

The closed-form solution of the above minimization problem [Aitken, 1936; Rao, 1945] is given by

$$\hat{\theta}_{\text{IWOLS}} = (\mathbf{X}^\top M_\omega \mathbf{X})^{-1} \mathbf{X}^\top M_\omega \mathbf{Y} \quad (3.10)$$

Gradient Descent

Similar to the OLS, we introduce the importance weighted gradient descent algorithm with an initial parameter, $\hat{\theta}_0 = 0$ and step size, γ . For $t = 1, \dots, T - 1$, the gradient is iteratively defined as:

$$\begin{aligned} \nabla \hat{\mathcal{R}}(\theta) &= \frac{2}{n} \mathbf{X}^\top M_\omega (\mathbf{X}\theta - \mathbf{Y}) \\ &= \frac{2}{n} \mathbf{X}^\top M_\omega \mathbf{X}\theta - \frac{2}{n} \mathbf{X}^\top M_\omega \mathbf{Y} \end{aligned} \quad (3.11)$$

and the estimated parameter after $t + 1$ iterations is given by:

$$\begin{aligned} \hat{\theta}_{t+1} &= \hat{\theta}_t - \gamma \nabla \hat{\mathcal{R}}(\hat{\theta}_t) \\ &= (\mathbf{I}_d - \gamma \cdot \mathbf{X}^\top M_\omega \mathbf{X}) \hat{\theta}_t + \frac{\gamma}{n} \mathbf{X}^\top M_\omega \mathbf{Y} \end{aligned} \quad (3.12)$$

3.1.2 Self-Attention Mechanism

Now we explore another method to estimate the *importance weights*: *self-attention mechanism*. Here, the model learns to assign different importance to different training samples dynamically and we treat the importance weights $\omega(X)$ as a trainable parameter just like the regression parameter, θ . The key idea here is that instead of assuming a fixed set of importance weights, we make them trainable parameters and optimize them iteratively.

That means, in the self-attention mechanism, we learn the weights jointly with the regression parameter and perform a gradient descent-ascent algorithm. We minimize the empirical loss with respect to the unknown regression parameters (same as before) and we maximize the empirical loss with respect to the weights. We take *gradient ascent* for learning weights because if we directly minimize the loss with respect to the importance weights, the model may reduce all weights to zero. Instead, we maximize the loss with respect to ω so that the model learns optimal importance weights that help in adjusting for covariate shift.

Let the empirical risk be given by

$$\hat{\mathcal{R}}(\theta, \omega) := \frac{1}{n} \sum_{i=1}^n m(\omega^{(i)}) \ell(y_i, \langle \theta, x_i \rangle) \quad (3.13)$$

where $\omega = (\omega^{(1)}, \dots, \omega^{(n)})$ is the importance weights for each sample and $m : \mathbb{R} \rightarrow \mathbb{R}$ is a masking function.

3.1. Importance Weighting

The gradient descent-ascent algorithm with initial regression parameter, $\theta_0 = 0$ and initial weights, $\omega_0^{(i)}$ for all $i = 1, \dots, n$ is given by:

$$\begin{aligned}\theta_{t+1} &= \theta_t - \gamma_t \nabla_{\theta} \hat{\mathcal{R}}(\theta_t, \omega_t) \\ \omega_{t+1} &= \omega_t - \eta_t \nabla_{\omega} \hat{\mathcal{R}}(\theta_t, \omega_t)\end{aligned}\tag{3.14}$$

where γ_t and η_t are the learning rates.

The empirical risk in equation 3.13 can be rewritten in matrix form as

$$\begin{aligned}\hat{\mathcal{R}}(\theta, \omega) &= \frac{1}{n} \|\mathbf{X}\theta - \mathbf{Y}\|_{\mathbf{M}}^2 \\ &= \frac{1}{n} (\mathbf{X}\theta - \mathbf{Y})^{\top} \mathbf{M} (\mathbf{X}\theta - \mathbf{Y})\end{aligned}\tag{3.15}$$

where $\mathbf{M} = \text{diag}(m(\omega^{(1)}), m(\omega^{(2)}), \dots, m(\omega^{(n)})) \in \mathbb{R}^{n \times n}$ is a diagonal matrix containing the weight values.

We need to compute the gradients of the above empirical risk with respect to ω and θ . This can be derived as:

$$\begin{aligned}\nabla_{\theta} \hat{\mathcal{R}}(\theta, \omega) &= \frac{2}{n} \mathbf{X}^{\top} \mathbf{M} (\mathbf{X}\theta - \mathbf{Y}) \\ \nabla_{\omega} \hat{\mathcal{R}}(\theta, \omega) &= \frac{1}{n} \frac{dM(\omega)}{d\omega} (\mathbf{X}\theta - \mathbf{Y}) \odot (\mathbf{X}\theta - \mathbf{Y})\end{aligned}\tag{3.16}$$

where $\frac{dM(\omega)}{d\omega} = \text{diag}\left(\frac{dm(\omega^{(j)})}{d\omega^{(j)}}\right)$ and $\odot$ represents the element-wise (*Hadamard*) product.

We can plug the values of $\nabla_{\theta} \hat{\mathcal{R}}(\theta, \omega)$ and $\nabla_{\omega} \hat{\mathcal{R}}(\theta, \omega)$ from equation 3.16 back in equation 3.14 to get the values of parameter θ and weights ω in each iteration.

Definition 3.1.1 (Masking Function). The masking function *modifies how the importance weights are applied* to the loss function. Possible choices are:

- *Unbounded masking*: $m_q(t) = t^q$ (for some $q \geq 0$)
 - If $q > 1$, larger weights are emphasized more
 - If $q = 0$, the weights are ignored (no importance weighting)
- *Bounded masking*: $m(t) = \tanh(t)$
 - Ensures that the weights remain within a limited range, preventing extreme values.

Choice of Masking Function

The ideal value of q in the masking function $m_q(t) = t^q$ depends on the balance between stability and adaptability of the learned importance weights ω :

- $q = 0$: No importance weighting effect, i.e., the model ignores importance weighting and assumes the train and test distributions are the same.

- $0 < q < 1$: Smoother weighting, prevents extreme values, i.e., the smaller weights are not suppressed too much, and larger weights are not overly emphasized.
- $q = 1$: Linear scaling of weights, which directly influence the loss function.
- $q > 1$: Large values of ω get amplified, giving more weight to high-importance samples. This can be useful if the shift is extreme, meaning some test samples are highly underrepresented in training. However, it may lead to instability, where a few samples dominate the training process.
- Very high q (e.g., $q \geq 3$): Risk of instability and overfitting. If a sample gets an unusually high importance weight, it overpowers the entire learning process.

Many studies suggest $q \approx 1$ or slightly lower (e.g., $q \in [0.5, 1]$) works well for general covariate shift scenarios. If the sample size is large, higher values of q (e.g., $1 \leq q \leq 2$) may work better since the weight distribution is smoother. If the sample size is small, lower values of q (e.g., $0.5 \leq q < 1$) prevent individual samples from dominating. If importance weights are estimated with noise, a bounded function like $m(t) = \tanh(t)$ may be preferable over any power function.

Choice of Initial Weight

Choice of $\omega_0^{(i)}$ for all $i = 1, \dots, n$ is very crucial for the stability and convergence of the gradient descent-ascent algorithm. If the initial weights are 0, no matter what masking function we select (unbounded or bounded), the masked weights will also be 0. Now, as we observe from equation 3.15 and 3.16, both gradients of the risk function (with respect to θ and ω) depends on the masked weights. An initial weights of 0 means these gradients are also 0 which means parameter θ and ω will not be updated and will be constant across iterations, as the second parts in equation 3.14 becomes zero.

Therefore, instead of 0, we initiate the initial weights $\omega_0^{(i)}$ for all $i = 1, \dots, n$ with very low value to avoid getting stuck.

Advantages of Self-attention Mechanism

Self-attention mechanism has several benefits compared to kernel density estimation or similar methods for estimating importance weights:

- *No need for additional data:*
 - Traditional importance weighting requires estimating density ratios using separate data or assumptions about distributions, whereas, this method provides a data-driven way to adjust for covariate shift without requiring separate density estimation.
 - This method learns the importance weights directly during model training.
- *Dynamic adaptation:*
 - Since weights are updated iteratively, they adapt based on the learning process, potentially leading to better generalization.

3.1. Importance Weighting

- *Handles covariate shift more effectively:*
 - Instead of assuming a fixed distribution shift, this approach lets the model learn the best way to correct for it.

3.1.3 Kernel Mean Matching (KMM)

Kernel Mean Matching [Gretton et al., 2009] directly estimates the importance weights without requiring explicit density estimation. Instead, it finds a set of weights β_i such that:

$$\sum_i \beta_i \phi(x_i) \approx \sum_j \phi(x_j) \quad (3.17)$$

where, $\phi : \mathcal{X} \rightarrow \mathcal{H}$, represents the feature mapping used in kernel methods, which maps input data x_i from the original feature space into a high-dimensional reproducing kernel Hilbert space (RKHS), which we already discussed in section 2.1.

This optimization ensures that the empirical kernel mean of the training data matches that of the test data, minimizing the distributional discrepancy in RKHS.

3.1.4 Logistic Regression-based Density Ratio Estimation

Here, we train a logistic regression model to distinguish between samples from the training and test distributions. The model is trained using labeled data, where samples from the training distribution are assigned one label and samples from the test distribution are assigned another. The estimated probabilities from the logistic regression model can then be used to approximate the density ratio between the two distributions [Qin and Lawless, 1994]. This method is straightforward and works well when the two distributions are not too different, but it may struggle in high-dimensional settings.

3.1.5 Kullback-Leibler Importance Estimation Procedure (KLIEP)

We can also directly estimate the density ratio by minimizing the Kullback-Leibler (KL) divergence between the reweighted training distribution and the test distribution. KLIEP formulates the problem as an optimization task where the density ratio function is learned without explicitly estimating the densities of the two distributions [Sugiyama et al., 2008]. This approach avoids density estimation and directly finds the importance weights, making it computationally efficient.

3.1.6 Least-Squares Importance Fitting (LSIF)

In LSIF, we estimate the density ratio function ω by minimizing a squared loss function between the estimated and true density ratios [Kanamori et al., 2009]. Instead of directly computing probability densities, LSIF uses a regression-like approach to fit the density ratio function in a way that minimizes the squared error. This method is computationally efficient and avoids density estimation altogether. Compared to KLIEP, LSIF often exhibits better numerical stability and is particularly useful in scenarios where importance weighting is required for domain adaptation or sample reweighting.

In general, importance weighting is very crucial when the model is parametric and misspecified. On the other hand, if the model is well-specified or over-parameterized, importance weighting may not be necessary [Gogolashvili et al., 2023].

3.2 Kernel Ridge Regression under Covariate Shift

Let us consider *Kernel Ridge Regression (KRR)*, a kernel-based method for regression that minimizes a regularized empirical loss (with the *least square loss*). Without covariate shift, the solution $f \in \mathcal{H}$ is found by solving:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \left\{ \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2 \right\}, \quad (3.18)$$

where $\lambda > 0$ is a regularization parameter that controls overfitting.

By the *representer theorem* [Kimeldorf and Wahba, 1971], the solution $f(x)$ can be written as a linear combination of kernel functions:

$$f(x) = \sum_{i=1}^n \alpha_i K(x, x_i) \quad (3.19)$$

where $K(x, x_i)$ is the kernel function.

Solving for α leads to the matrix formulation:

$$\hat{\alpha} = (K + \lambda \mathbf{I})^{-1} Y \quad (3.20)$$

To adjust for the covariate shift, we apply the importance weights to the loss function:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \left\{ \frac{1}{n} \sum_{i=1}^n \omega(x_i) (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2 \right\} \quad (3.21)$$

The solution to this problem can be written in terms of the kernel matrix $K \in \mathbb{R}^{n \times n}$ with entries $K_{ij} = k(X_i, X_j)$, and the importance weights can be incorporated into a diagonal matrix $W = \text{diag}(\omega(x_1), \dots, \omega(x_n))$. The closed-form solution for the KRR estimator under covariate shift is given by:

$$\hat{f} = (KWK + \lambda \mathbf{I})^{-1} K W Y, \quad (3.22)$$

where $Y = (y_1, \dots, y_n)^\top$ is the vector of training labels. This solution adjusts for the shift by weighting the kernel matrix and labels according to the test distribution.

Once $\omega(X)$ is estimated, it can be plugged into the weighted KRR formulation to adjust for the covariate shift.

KRR achieves minimax optimal rates under covariate shift when the density ratio is bounded, but naive empirical risk minimization fails to generalize in such settings [Ma et al., 2022]. Furthermore, when the ratio is unbounded but has a finite second moment, a reweighted KRR with a truncated likelihood ratio yields the best possible estimation rates.

3.3 Alternative Strategies

While importance weighting and Kernel Ridge Regression are very common solutions, several alternative approaches exist to tackle the problem of covariate shift:

3.3.1 Domain Adaptation

Rather than reweighting training samples, domain adaptation techniques aim to learn domain-invariant representations. One of the most popular kernel-based domain adaptation techniques includes: *Maximum Mean Discrepancy (MMD) Regularization*.

MMD is a measure of discrepancy between two distributions in RKHS [Gretton et al., 2012]:

$$\text{MMD}(\mathbb{P}_{\text{train}}, \mathbb{P}_{\text{test}}) = \|\mathbb{E}_{\mathbf{X} \sim \mathbb{P}_{\text{train}}}[\phi(\mathbf{X})] - \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_{\text{test}}}[\phi(\mathbf{X})]\|_{\mathcal{H}} \quad (3.23)$$

By adding MMD as a regularization term to the kernel learning objective:

$$\min_f \sum_i \ell(f(x_i), y_i) + \lambda \text{MMD}(\mathbb{P}_{\text{train}}, \mathbb{P}_{\text{test}}), \quad (3.24)$$

one encourages the feature space representations of train and test distributions to align, making the learned model more robust to distributional shifts since it forces the distributions to be similar in RKHS.

Besides MMS regularization, *Adversarial Training* is another popular domain adaptation technique, which also encourages domain-invariant feature learning.

3.3.2 Transfer Learning

Transfer learning leverages knowledge from a source domain to improve learning in the target domain. Kernel-based transfer learning approaches include *Adaptive regularization using RKHS norms* [Bach, 2024] and *Subspace alignment techniques using kernel PCA*, where we map training and test distributions into a shared feature space.

3.3.3 Robust Risk Minimization in RKHS

Instead of reweighting, another approach is to explicitly optimize for robustness against worst-case distribution shifts. In the kernel setting, this involves solving a distributionally robust optimization (DRO) problem [Rahimian and Mehrotra, 2019]:

$$\min_f \sup_{Q \in \mathcal{U}(\mathbb{P}_{\text{train}})} \mathbb{E}_Q[\ell(f(X), Y)], \quad (3.25)$$

where $\mathcal{U}(\mathbb{P}_{\text{train}})$ is an uncertainty set around the training distribution, i.e., the set $\mathcal{U}(\mathbb{P}_{\text{train}})$ consists of distributions that are considered close to the training distribution $\mathbb{P}_{\text{train}}$. Kernel methods adapt DRO by defining $\mathcal{U}$ using RKHS-based distances like MMD.

4 Supervised Learning under Covariate Shift

Let us explore the problem of covariate shift under the settings of supervised learning.

Let $\mathcal{S} = \{(x_i, y_i)\}_{i=1}^n$ be the training samples, where $X = \{x_i\} \in \mathcal{X} \subset \mathbb{R}^d$ is i.i.d. training samples following a probability distribution $\mathbb{P}_{\text{train}}(X)$ and $Y = \{y_i\} \in \mathcal{Y} \subset \mathbb{R}$ is a corresponding training output value following a conditional probability distribution $\mathbb{P}(Y | X)$.

Let $\ell(y, \hat{y})$ be the *loss function*, which measures the discrepancy between the true output value y at an input point x and its estimate $\hat{y}$. Let us employ a parametric model $\hat{f}(x; \theta)$ for estimating the output value y , where $\theta \in \mathbb{R}^d \subset \Theta$ (a parameter space). A model $\hat{f}(x; \theta)$ is said to be *correctly specified* if there exists a parameter θ^* such that $\hat{f}(x; \theta^*) = f(x)$; otherwise the model is said to be *misspecified*.

4.1 Goal of Supervised Learning Algorithms

Let us consider a test sample, which is not known during the training phase, and we denote this test sample by $(\tilde{X}, \tilde{Y}) \in \mathcal{T}$, where $\tilde{X}$ are the test input points and $\tilde{Y}$ are the corresponding test output values, both of which are unknown during the training phase.

In a traditional supervised learning algorithm (regression or classification), we try to estimate an unknown relationship between input and output variables from the training samples $\mathcal{S} = \{(x_i, y_i)\}_{i=1}^n$, which we can use to predict a real-valued response $\hat{y}_t$ based on a vector of covariates $\tilde{x}_t$ from the test dataset $\mathcal{T}$.

In other words, the goal of supervised learning is to determine the value of the parameter θ from the training samples, so that output values for unlearned test input points are estimated *as accurately as possible* i.e., $\hat{y}_t = f(\tilde{x}_t; \hat{\theta}) \approx \tilde{y}_t$ for all $(\tilde{x}_t, \tilde{y}_t) \in \mathcal{T}$. Here, $\hat{\theta}$ is the estimated (learned) parameter, which generally depends on $\mathcal{S} = \{(x_i, y_i)\}_{i=1}^n$.

We assume that the pair (X, Y) has an unknown distribution $\mathbb{P}$ and X has marginal distribution $\mathbb{P}_X$ (we refer to it as the *source distribution*). We aim to learn the parameter θ by minimizing the risk function:

$$f^* \in \arg \min_f \mathcal{R}_{\mathbb{P}}(f) \tag{4.1}$$

4.2. Introducing Covariate Shift to the Model

where,

$$\mathcal{R}_{\mathbb{P}}(f) = \mathbb{E}_{\mathbb{P}}[\ell(y, f(x; \theta))] \quad (4.2)$$

4.2 Introducing Covariate Shift to the Model

In standard supervised learning theories, the test sample is assumed to follow $\mathbb{P}(Y | X)\mathbb{P}_{\text{train}}(X)$, which is the same probability distribution as for the training samples $\mathcal{S} = \{(X, Y)\} = \{(x_i, y_i)\}_{i=1}^n$ [Sugiyama et al., 2000]. However, in what follows, we consider the situation under *covariate shift* what we discussed before in section 2, i.e., when

1. the conditional distribution $\mathbb{P}(Y | X)$ remains unchanged across train and test sample (conditional distribution stability), but
2. the test input points follows a probability distribution $\mathbb{P}_{\text{test}}$ different than $\mathbb{P}_{\text{train}}$ (input distribution change).

In other words, it follows:

$$\mathbb{P}_{\text{train}}(Y | X) = \mathbb{P}_{\text{test}}(Y | X) \quad (4.3)$$

but

$$\mathbb{P}_{\text{train}}(X) \neq \mathbb{P}_{\text{test}}(X) \quad (4.4)$$

Now, let $p_{\text{train}}(X)$ and $p_{\text{test}}(X)$ be the probability density functions corresponding to the input distribution $\mathbb{P}_{\text{train}}(X)$ and $\mathbb{P}_{\text{test}}(X)$, respectively. $\mathbb{P}_{\text{train}}(X)$ and $\mathbb{P}_{\text{test}}(X)$ are also called *source distribution* and *target distribution* respectively. For theoretical purposes, we can assume that the ratio of test and training input densities at training input points

$$\omega(x_i) = \frac{p_{\text{test}}(x_i)}{p_{\text{train}}(x_i)} = \frac{d\mathbb{P}_{\text{test}}(x_i)}{d\mathbb{P}_{\text{train}}(x_i)} \quad (4.5)$$

is finite and known, where $\omega : \mathcal{X} \rightarrow \mathbb{R}_+$ is the *Radon-Nikodym derivative* of the target measure with respect to the source measure. We refer the above concept as *importance sampling* [Fishman, 1996] and the individual weight ratios are called *importance weight*.

Importance weights $\omega(x_i)$ can also be expressed as a factor to reweight the test sample points:

$$d\mathbb{P}_{\text{test}}(x_i) = w(x_i)d\mathbb{P}_{\text{train}}(x_i) \quad (4.6)$$

Further, we can assume [Ma et al., 2022] that either of

1. $\sup_x |w(x)| \leq B$ i.e., uniforml bounded or
2. $\mathbb{E}_{\mathbb{P}_{\text{train}}}[w(x)^2] \leq \tau^2, \tau^2 \geq 1$

holds as the bound of $\omega(x)$ with condition (1) being the stricter one.

In practical situations, the importance weights are usually unknown, since we do not know the underlying distributions. So we replace them with the empirical estimates, which we have explained in section 3.1.

Example 4.2.1 (Regression). For each fixed $X = x \in \mathcal{X}$, the *optimal* estimator for a mean-squared loss function is given by the regression function $f^*(x) := \mathbb{E}[y \mid X = x]$, that is,

$$f^* \in \arg \min_{f \in L^2(\mathcal{X}, \mathbb{P}_X)} \mathcal{R}_{\mathbb{P}}(f) \quad (4.7)$$

where,

$$\mathcal{R}_{\mathbb{P}}(f) = \mathbb{E}_{\mathbb{P}}[(Y - f(X))^2] \quad (4.8)$$

It can be noted that for all $f \in L^2(\mathcal{X}, \mathbb{P}_X)$, the excess risk is given by

$$\mathcal{R}_{\mathbb{P}}(f) - \mathcal{R}_{\mathbb{P}}(f^*) = \|f - f^*\|_{L^2(\mathbb{P}_X)}^2 \quad (4.9)$$

To find an estimator $\hat{f}_{\mathcal{S}}$ for f^* , we can make use of the i.i.d training samples $\mathcal{S} = \{(x_1, y_1), \dots, (x_n, y_n)\} \in (\mathcal{X} \times \mathcal{Y})^n$ i.e., $\mathcal{S} \sim \mathbb{P}^n$.

In case of covariate shift, the *target distribution* $\mathbb{P}_{\text{test}}$ is different from the source distribution $\mathbb{P}_{\text{train}}$, which means the training and test samples follow different distributions. So, our goal is to find an estimator $\hat{f}_{\mathcal{S}}$ trained on $\mathcal{S} \sim \mathbb{P}_{\text{train}}^n$ whose excess risk with respect to $\mathbb{P}_{\text{test}}$ is small, i.e.,

$$\mathcal{R}_{\mathbb{P}_{\text{test}}}(\hat{f}_{\mathcal{S}}) - \mathcal{R}_{\mathbb{P}_{\text{test}}}(f^*) = \|\hat{f}_{\mathcal{S}} - f^*\|_{L^2(\mathbb{P}_{\text{test}})}^2 \quad (4.10)$$

is small.

Example 4.2.2 (Mean Shift). Let us consider a special case of covariate shift - "*mean shift*" in the context of the family of multivariate normal distributions:

$$\mathbb{P}_{\text{train}} = \mathcal{N}(0, \Sigma), \quad \mathbb{P}_{\text{test}} = \mathcal{N}(\mu, \Sigma), \quad (4.11)$$

where $\mu \in \mathbb{R}^d$ is the mean and positive definite $\Sigma : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is the covariance matrix. As we recall, the probability density function (PDF) of a multivariate normal (Gaussian) distribution with mean vector μ and covariance matrix Σ for a d -dimensional random variable X is given by:

$$f_{\mu, \Sigma}(X) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (X - \mu)^{\top} \Sigma^{-1} (X - \mu) \right) \quad (4.12)$$

Hence, the weights can be calculated directly as:

$$\omega(X) = e^{\frac{1}{2}(\mu^{\top} \Sigma^{-1} \mu + 2x^{\top} \Sigma^{-1} \mu)} \quad (4.13)$$

However, they are not bounded.

4.3 Empirical Risk Minimization and its Importance Weighted Variants

4.3.1 Empirical Risk Minimization

Once again, we recall the goal of supervised learning: Given some observations $(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$, $(i = 1, 2, \dots, n)$ of inputs and outputs (or features/variables) that we call *training*

4.3. Empirical Risk Minimization and its Importance Weighted Variants

data and given a new $\tilde{x} \in \mathcal{X} = \mathbb{R}^d$, predict $\tilde{y} \in \mathcal{Y} = \mathbb{R}$ (*test data*) with a regression function $f : \mathcal{X} \rightarrow \mathcal{Y}$ such that $f(\tilde{x}) \approx \tilde{y}$.

Considering a mean-square loss function, $\ell(y_i, \hat{y}_i) = \frac{1}{n} \sum_{i=1}^n (y_i - f_\theta(x_i))^2$, where y_i is the true output and $\hat{y}_i$ is the predicted output, the empirical risk for the regression case is defined by:

$$\hat{\mathcal{R}}(\theta) := \frac{1}{n} \sum_{i=1}^n (y_i - f_\theta(x_i))^2, \theta \in \Theta \quad (4.14)$$

which we compute on the given observations $(x_i, y_i), i = 1, 2, \dots, n$. To obtain a "good" or "reasonable" estimator, we minimize the empirical risk, which is known as *empirical risk minimization* (ERM), which is a standard method to learn the parameter θ in any supervised learning algorithm [Vapnik, 1998; Schölkopf and Smola, 2002]. That means, we try to learn the parameter θ , for which the above risk (in 4.14) is minimum:

$$\hat{\theta}_{\text{ERM}} = \arg \min_{\theta \in \Theta} \left[\frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{f}(x_i; \theta)) \right] \quad (4.15)$$

where $\ell : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ is a loss function.

In traditional cases of supervised learning, when $\mathbb{P}_{\text{train}}(X) = \mathbb{P}_{\text{test}}(X)$ i.e. the train and test distributions are the same, ERM provides a consistent estimator [Shimodaira, 2000]. However, under the conditions of covariate shift (as explained earlier in section 2), where $\mathbb{P}_{\text{train}}(X) \neq \mathbb{P}_{\text{test}}(X)$, ERM is not generally consistent anymore [Shimodaira, 2000]:

$$\lim_{n \rightarrow \infty} (\hat{\theta}_{\text{ERM}}) \neq \theta^* \quad (4.16)$$

where θ^* is the *true optimal model parameter* that minimizes the expected loss under the test distribution:

$$\theta^* = \arg \min_{\theta \in \Theta} \left(\mathbb{E}_{(\tilde{x}, \tilde{y}) \sim \mathcal{T}} \left[\ell(\tilde{y}, \hat{f}(\tilde{x}, \theta)) \right] \right) \quad (4.17)$$

4.3.2 Importance Weighted Variant of ERM

Under the conditions of covariate shift, we use the following *importance weighted empirical risk minimization* (IWERM), which gives us more consistent results [Shimodaira, 2000]:

$$\hat{\theta}_{\text{IWERM}} = \arg \min_{\theta \in \Theta} \left[\frac{1}{n} \sum_{i=1}^n \frac{p_{\text{test}}(x_i)}{p_{\text{train}}(x_i)} \ell(y_i, \hat{f}(x_i; \theta)) \right] \quad (4.18)$$

We can use a simpler representation using the notation of $\omega(x_i)$ as:

$$\hat{\theta}_{\text{IWERM}} = \arg \min_{\theta \in \Theta} \left[\frac{1}{n} \sum_{i=1}^n w(x_i) \ell(y_i, \hat{f}(x_i; \theta)) \right] \quad (4.19)$$

This satisfies even for a misspecified model:

$$\lim_{n \rightarrow \infty} (\hat{\theta}_{\text{IWERM}}) = \theta^* \quad (4.20)$$

Even with the increased consistency, IWERM is not always efficient and can be rather unstable in certain cases [Shimodaira, 2000]. Therefore, in practical cases with *finite* samples, IWERM turns out to be not optimal, and we can use a slightly stabilized variant of IWERM [Sugiyama et al., 2000].

4.3.3 Other Variants of Importance Weighted ERM

We aim to control the trade-off between consistency and stability either by (a) weakening the weights (in the case of *adaptive importance weighted empirical risk minimization* (AIWERM)) as:

$$\hat{\theta}_{\text{AIWERM}} = \arg \min_{\theta \in \Theta} \left[\frac{1}{n} \sum_{i=1}^n \left(\frac{p_{\text{test}}(x_i)}{p_{\text{train}}(x_i)} \right)^\lambda \ell(y_i, \hat{f}(x_i; \theta)) \right] \quad (4.21)$$

which, in simpler terms using the notation $\omega(x_i)$, is:

$$\hat{\theta}_{\text{AIWERM}} = \arg \min_{\theta \in \Theta} \left[\frac{1}{n} \sum_{i=1}^n w(x_i)^\lambda \ell(y_i, \hat{f}(x_i; \theta)) \right] \quad (4.22)$$

or (b) adding a regularization term to the empirical risk (in the case of *regularized importance weighted empirical risk minimization* (RIWERM)) as:

$$\hat{\theta}_{\text{IWERM}} = \arg \min_{\theta \in \Theta} \left[\frac{1}{n} \sum_{i=1}^n \frac{p_{\text{test}}(x_i)}{p_{\text{train}}(x_i)} \ell(y_i, \hat{f}(x_i; \theta)) + \gamma \Omega(\theta) \right] \quad (4.23)$$

which, in simpler terms using the notation $\omega(x_i)$, is:

$$\hat{\theta}_{\text{IWERM}} = \arg \min_{\theta \in \Theta} \left[\frac{1}{n} \sum_{i=1}^n w(x_i) \ell(y_i, \hat{f}(x_i; \theta)) + \gamma \Omega(\theta) \right] \quad (4.24)$$

where $0 \leq \lambda \leq 1$, $\gamma \geq 0$, and $\Omega(\theta)$ are some regularization functions.

In addition to above AIWERM and RIWERM methods, there can be many other possibilities of controlling the trade-off between consistency and stability to learn the parameter θ , which are based on the same methodology. For example, an importance weighted variant of support vector machines [Vapnik, 1998; Schölkopf and Smola, 2002; Huang et al., 2007] or graph regularization techniques [Bousquet et al., 2004; Belkin and Niyogi, 2004; Hein, 2006] can also be employed.

5 State of the Art in Covariate Shift Regularization

Covariate shift, a fundamental challenge in statistical learning, which occurs when:

1. the distribution of input features differs between training and test datasets

$$\mathbb{P}_{\text{train}}(X) \neq \mathbb{P}_{\text{test}}(X)$$

while

2. the conditional distribution of the target given the input remains unchanged.

$$\mathbb{P}_{\text{train}}(Y | X) = \mathbb{P}_{\text{test}}(Y | X)$$

In supervised learning algorithms, various recent studies have advanced our understanding of effective adaptation strategies under covariate shift. While Reproducing Kernel Hilbert Space (RKHS)-based methods have received significant attention due to their strong theoretical foundation, other approaches including neural networks and decision trees, which have been studied in the context of covariate shift adaptation, have also shown promising results. For instance, neural networks have been investigated with domain adaptation techniques such as adversarial training, while decision trees leverage reweighting methods to adjust for distributional differences.

5.1 When Is Importance Weighting Necessary?

[Gogolashvili et al., 2023] investigated scenarios where importance weighting enhances learning performance. Importance weighting is particularly useful when the model is misspecified. In such cases, accounting for distributional differences through weighting improves estimation accuracy and mitigates biases introduced by the different distribution of the training set. Given a source distribution $\mathbb{P}_{\mathcal{S}}(X)$ and a target distribution $\mathbb{P}_{\mathcal{T}}(X)$, the expected risk under importance weighting [Sugiyama et al., 2000] is given by:

$$\mathcal{R}_w(\theta) = \mathbb{E}_{\mathbb{P}_{\mathcal{T}}}[\ell(Y, f(X; \theta))] = \mathbb{E}_{\mathbb{P}_{\mathcal{S}}}[\omega(X)\ell(Y, f(X; \theta))] \quad (5.1)$$

where $w(X) = \frac{d\mathbb{P}_{\mathcal{T}}(X)}{d\mathbb{P}_{\mathcal{S}}(X)}$ is the importance weight.

However, when the model is already well-specified, the necessity of importance weighting diminishes, especially as sample size increases. For large sample sizes, direct kernel ridge regression (KRR) without reweighting can achieve near-optimal performance, making importance weighting less critical.

5.2 Optimal Learning Under Covariate Shift

[Ma et al., 2022] explored minimax-optimal learning strategies under covariate shift and demonstrated that Kernel ridge regression (KRR) combined with importance weighting attains the minimax rate when the density ratio function $\omega(X)$ is bounded, ensuring stable learning despite shifts in data distribution. When $\omega(X)$ is unbounded but possesses a finite second moment, truncated importance weighting provides optimal learning rates by preventing extreme weight values from destabilizing model training. The optimal weighted KRR estimator is given by:

$$\hat{f} = \arg \min_{f \in \mathcal{H}_K} \sum_{i=1}^n \omega(x_i) \ell(y_i, f(x_i; \theta))^2 + \lambda \|f\|_{\mathcal{H}_K}^2 \quad (5.2)$$

where $\mathcal{H}_K$ is the RKHS associated with kernel K , and λ is the regularization parameter.

5.3 RKHS-based Theoretical Analysis and Generalization Bounds

[Bach, 2024] and [Mohri et al., 2018] extended theoretical analyses of RKHS-based learning under distribution shifts. A well-regularized kernel model within RKHS frameworks enhances model generalization under covariate shift and exhibits stable out-of-sample performance by controlling the complexity of the function class and preventing overfitting to the biased training data.

Properly chosen kernel parameters can significantly reduce sensitivity to covariate shift, reinforcing the role of regularization in robust machine learning. Regularization techniques, such as norm control in RKHS, play a pivotal role in mitigating performance degradation under distributional shifts. The RKHS norm control is achieved through:

$$\min_{f \in \mathcal{H}_K} \|f\|_{\mathcal{H}_K}^2 \quad \text{subject to} \quad \mathbb{E}_{\mathbb{P}_T}[\ell(y, f(x; \theta))] \leq \epsilon \quad (5.3)$$

ensuring bounded model complexity under covariate shift. A generalization bound for learning under covariate shift is given by:

$$\mathcal{R}_{\mathbb{P}_T}(f) \leq \mathcal{R}_{\mathbb{P}_S}(f) + \mathcal{O} \left(\sqrt{\frac{\mathbb{E}_{\mathbb{P}_S}[w(x)^2]}{n}} \right) \quad (5.4)$$

where $\mathcal{R}_{\mathbb{P}_T}(f)$ and $\mathcal{R}_{\mathbb{P}_S}(f)$ are the risks under target and source distributions, respectively.

5.4 Covariate Shift Adaptation via Importance-Weighted Cross-Validation

[Sugiyama et al., 2000] pioneered the use of importance-weighted cross-validation (IWCV) as a method for adapting to covariate shift. IWCV can be an effective technique for model selection under covariate shift, ensuring that models generalize well to the test distribution. It was demonstrated that IWCV significantly improves model selection accuracy compared to standard cross-validation, particularly in scenarios with large distribution discrepancies between training and test sets. To compensate for the effect of the covariate shift in the standard CV procedure, following importance weighted cross validation (IWCV) variant was proposed

$$\hat{R}_{\text{kIWCV}}^{(n)} = \frac{1}{k} \sum_{i=1}^k \frac{1}{|\mathcal{T}_i|} \sum_{(x,y) \in \mathcal{T}_i} w(x) \ell(y, \hat{f}_{\mathcal{T}_i}(x)) \quad (5.5)$$

and

$$\hat{R}_{\text{LOOIWCV}}^{(n)} = \frac{1}{n} \sum_{i=1}^n w(x_i) \ell(y_i, \hat{f}_i(x_i)) \quad (5.6)$$

for K-fold cross validation and Leave-One-Out cross validation, respectively. LOOIWCV gives an almost unbiased estimate of the risk even under the covariate shift.

5.5 Covariate Shift Adaptation in Neural Networks and Decision Trees

Recent studies have explored methods for handling covariate shift in neural networks and decision trees, two widely used supervised learning models. These approaches leverage domain adaptation techniques, reweighting strategies, and adversarial learning to improve generalization across shifting distributions.

5.5.1 Neural Networks and Domain Adaptation

Neural networks are particularly susceptible to distribution shifts due to their reliance on large-scale training data. Several methods have been proposed to handle this challenge, with adversarial domain adaptation being one of the most effective techniques. [Ganin et al., 2016] introduced the *Domain-Adversarial Neural Network (DANN)*, which minimizes domain discrepancy by learning domain-invariant features using a domain classifier. Given a source distribution $\mathbb{P}_{\mathcal{S}}(X)$ and a target distribution $\mathbb{P}_{\mathcal{T}}(X)$, adversarial training minimizes the domain discrepancy by optimizing:

$$\min_f \min_D \mathbb{E}_{\mathbb{P}_{\mathcal{S}}}[\log D(f(X))] + \mathbb{E}_{\mathbb{P}_{\mathcal{T}}}[\log(1 - D(f(X)))] \quad (5.7)$$

where $f(X)$ is the feature representation learned by the neural network, and $D(X)$ is a domain classifier that distinguishes between source and target distributions. The adversarial loss forces $f(x)$ to generate domain-invariant representations.

Another strategy is *importance-weighted training for deep networks*, where a weighted loss function is used:

$$\hat{\mathcal{R}}_{\omega}(\theta) = \sum_{i=1}^n \omega(x_i) \ell(y_i, f(x_i; \theta)) \quad (5.8)$$

where $\omega(x_i) = \frac{\mathbb{P}_{\mathcal{T}}(x_i)}{\mathbb{P}_{\mathcal{S}}(x_i)}$ accounts for distributional differences, allowing the model to focus more on examples similar to the test distribution.

5.5.2 Decision Trees and Reweighting Methods

Decision trees, including ensemble methods like Random Forests and Gradient Boosted Trees, often handle covariate shift using reweighting and resampling techniques.

- **Reweighting Training Instances:**

Similar to kernel methods, decision trees adjust for covariate shift by modifying instance weights. For a loss function $\ell(y, f(x; \theta))$, the weighted empirical risk is:

$$\mathcal{R}_w(\theta) = \sum_{i=1}^n w(x_i) \ell(y_i, f(x_i; \theta)) \quad (5.9)$$

ensuring that samples from the shifted test distribution receive higher influence.

- **Sample Reweighting via Bias Correction:**

[Sugiyama et al., 2000] proposed a bias correction method for tree-based models, where the training set is modified to match the test distribution using importance weighting.

- **Domain-Adaptive Decision Trees:**

[Zhao et al., 2021] introduced decision trees optimized for covariate shift by incorporating distributional constraints into the splitting criteria:

$$\text{Split criterion} = \text{Gini}(\mathcal{S}) - \lambda \cdot \mathbb{E}[w(x)] \quad (5.10)$$

where λ controls the impact of distribution weighting, ensuring that splits are more aligned with the test distribution and $\text{Gini}(\mathcal{S})$ is the Gini index (or Gini impurity) for the dataset $\mathcal{S}$. Gini index (or Gini impurity) is a measure of the impurity or uncertainty in a dataset, commonly used in decision trees for selecting the best feature to split on.

Recent research has advanced covariate shift adaptation across RKHS-based learning, neural networks, and decision trees. Importance weighting remains crucial for misspecified models and unbounded density ratios, while RKHS regularization and generalization bounds enhance robustness. Neural networks mitigate shift through adversarial adaptation and importance weighting, whereas decision trees leverage reweighting and modified splitting criteria. Combining these approaches with kernel methods provides a comprehensive framework for handling covariate shift. Future work should focus on scalable, adaptive strategies to improve generalization in real-world applications.

6 Experiment Setup

For our experimental setup, we will now generate data for both train and test sample under the conditions of covariate shift and show its effect on the model performance. We will also try to regularize the effect of covariate shift by importance weighting. In particular, we will use *kernel density estimation* and *self-attention mechanism* to estimate the weights.

6.1 Covariate Shift in Linear Regression

For our experiment and problem formulation, we focus on linear regression with a 10-dimensional feature space and a scalar outcome. So, $\mathcal{X} = \mathbb{R}^{10}$ and $\mathcal{Y} = \mathbb{R}$, and our observations follow the model:

$$y_i = f^*(x_i) + \epsilon_i, \quad f^*(x_i) = \langle \theta^*, x_i \rangle, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2) \quad (6.1)$$

with $i = 1, \dots, 1000$ and $\theta^* = \mathbf{1}^{10}$ (a vector of 1).

The training set $\mathcal{S} = \{(x_i, y_i)\}$ are independent and identically distributed (i.i.d) from $\mathbb{P}_{\text{train}} = \mathcal{N}(0, 1)$. We denote the design matrix by $\mathbf{X} \in \mathbb{R}^{1000 \times 10}$ and the vector of outputs by $\mathbf{Y} = (y_1, \dots, y_{1000})^\top \in \mathbb{R}^{1000}$.

We also draw multiple test dataset $\mathcal{T}_\mu = \{(\tilde{x}_t, \tilde{y}_t)\} \sim \mathcal{N}(\mu, 1)$ with $t = 1, \dots, 1000$ and $\mu \in \{0, 0.2, 0.4, \dots, 1.8, 2.0\}$.

That means, we have 11 test dataset, each with sample size = 1000 and *mean shift*. The multiple test dataset will help us understand the effect of varying mean shift in estimating the parameter $\hat{\theta} \in \mathbb{R}^{10}$ and the model performance.

Note: In practical situations, we do not know the underlying distributions of both train and test data. Here, for experimental purpose, we have generated the data following *Gaussian distribution*, with test data having *mean shift*.

Measure of Performance:

For our experiment, since we are focusing on linear regression, we have considered *mean square error (MSE)* as the *loss function* to measure the error in predicting the outcome.

For an estimator $\hat{f}(x) = \langle \hat{\theta}, x \rangle$, the mean square error on a test sample $(\tilde{x}_i, \tilde{y}_i)$ is measured as:

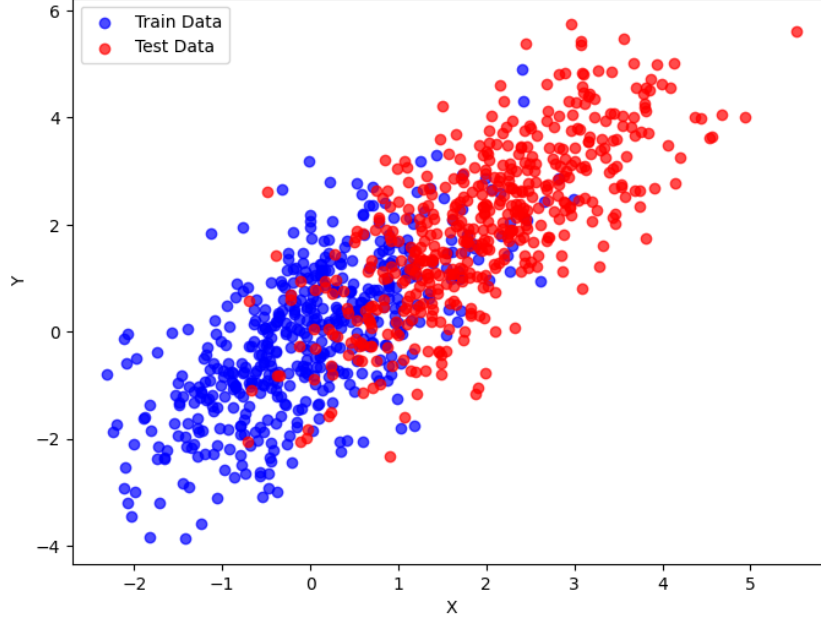


Figure 6.1: Visual example of training data and test data drawn from Gaussian distribution with mean shift. Here, the training sample (in blue) are drawn from $\mathcal{N}(0, 1)$ and the test sample (in red) are drawn from $\mathcal{N}(2, 1)$ each with a sample size of 500 and the underlying relation: $y_i = x_i + \epsilon_i$, $\epsilon_i \sim \mathcal{N}(0, 1)$, $i = 1, \dots, 500$. This is just for visual example of how covariate shift effects and in our experiment, we are not working on this data.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{f}(\tilde{x}_i) - \tilde{y}_i)^2 = \frac{1}{n} \sum_{i=1}^n \left(\langle \hat{\theta}, \tilde{x}_i \rangle - \tilde{y}_i \right)^2 \quad (6.2)$$

i.e., the average squared difference between the true value and predicted value of the outcome, where n is the number of samples in the test dataset.

6.2 Effect of Mean Shift on MSE

Now, we experiment how the mean shift μ affect the model performance in estimating the outcome using two most popular methods: *Ordinary Least Square (OLS)* and *Gradient Descent*.

6.2.1 Ordinary Least Square (OLS) Estimator

We briefly recall the definition of OLS for a *design matrix* $\mathbf{X} \in \mathbb{R}^{n \times d}$ below.

Definition 6.2.1 (Ordinary Least Squares). When the design matrix $\mathbf{X}$ has full column rank, i.e., $\text{rank}(\mathbf{X}) = d$ meaning, the problem is *overdetermined* and we have $n \geq d$ (more observations than feature dimensions), then the minimizer of the empirical risk (with least square loss function)

$$\mathcal{R}(\theta) = \frac{1}{n} \|\mathbf{Y} - \mathbf{X}\theta\|_2^2 \quad (6.3)$$

is as below:

6.2. Effect of Mean Shift on MSE

$$\hat{\theta}_{\text{OLS}} := \arg \min_{\theta \in \mathbb{R}} \mathcal{R}(\theta) = \arg \min_{\theta \in \mathbb{R}} \frac{1}{n} \|\mathbf{Y} - \mathbf{X}\theta\|_2^2 \quad (6.4)$$

and is called the *Ordinary Least Squares* estimator.

Proposition 6.2.1 (Closed form solution of OLS). If $\text{rank}(\mathbf{X}) = d$, we can show that $\hat{\theta}_{\text{OLS}}$ exists and is unique. It also attains a closed form solution [Aitken, 1936; Rao, 1945]:

$$\hat{\theta}_{\text{OLS}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y} \quad (6.5)$$

In our problem setting, we use this closed form solution to estimate the parameter θ and use them to predict the outcome for inputs $\tilde{x}$ from test dataset $\mathcal{T}_\mu$ for varying μ . Finally, we measure our performance by calculating MSE and plot the resulting MSE against the corresponding mean shift (μ).

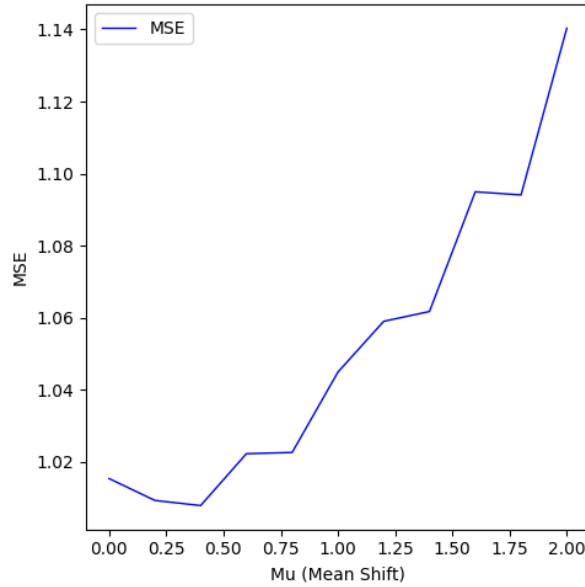


Figure 6.2: Effect of varying mean shift on Mean Square Error (MSE) in Ordinary Least Square (OLS) estimator setup

Figure 6.2 illustrates the effect of covariate shift on model performance using ordinary least squares (OLS) estimator. As the mean shift (μ) increases, the MSE also rises, indicating that the model's predictive accuracy deteriorates when the input distribution (test data) deviates from its training distribution. Initially, small shifts ($\mu < 0.5$) have a minor impact, but as μ grows, the error escalates more rapidly, suggesting a significant sensitivity to distributional changes. This behavior highlights the challenges posed by covariate shift.

6.2.2 Gradient Descent Estimator

Now, we will repeat the experiment with Gradient Descent with two different step size, $\gamma = 0.1$ and $\gamma = 0.05$, to also observe the effect of step size (γ) on convergence rate. We recall the definition gradient descent before.

Definition 6.2.2 (Gradient Descent). The gradient descent is an iterative method where in each iteration, it moves towards the direction of steepest descent to move closer to the minima [Cauchy, 1847]. It iterates with fixed step size $\gamma > 0$ and an initial parameter $\theta_0 \in \mathbb{R}^d$.

For a empirical risk with least-square loss function

$$\mathcal{R}(\theta) = \frac{1}{n} \|\mathbf{Y} - \mathbf{X}\theta\|_2^2 \quad (6.6)$$

the gradient is given by

$$\begin{aligned} \nabla \hat{\mathcal{R}}(\theta) &= \frac{2}{n} \mathbf{X}^\top (\mathbf{X}\theta - \mathbf{Y}) \\ &= \frac{2}{n} \mathbf{X}^\top \mathbf{X}\theta - \frac{2}{n} \mathbf{X}^\top \mathbf{Y} \end{aligned} \quad (6.7)$$

and the estimated parameter $\hat{\theta}$ after $t + 1$ iterations is given by

$$\begin{aligned} \hat{\theta}_{t+1} &= \hat{\theta}_t - \gamma \nabla \hat{\mathcal{R}}(\hat{\theta}_t) \\ &= (\mathbf{I}_d - 2\frac{\gamma}{n} \mathbf{X}^\top \mathbf{X}) \hat{\theta}_t + 2\frac{\gamma}{n} \mathbf{X}^\top \mathbf{Y} \end{aligned} \quad (6.8)$$

In our problem setting, we use $\theta_0 = \mathbf{0}^{10}$ (a vector of zero) and run the estimator for 250 iterations, i.e., $T_{\max} = 250$. We carry out the experiment using gradient descent for two different step sizes, $\gamma = 0.1$ and $\gamma = 0.05$.

Figure 6.3 illustrates the behavior of Mean Squared Error (MSE) using gradient descent method under different levels of covariate shift ($\mu = 0.2$ in left and $\mu = 2.0$ in right) and different step sizes, $\gamma = 0.1$ in top and $\gamma = 0.05$ in bottom. We observe that for gradient descent, the MSE initially starts at a high value and decreases rapidly as iterations increase, indicating convergence towards a minimum. However, for $\mu = 2.0$ (right), the initial MSE is significantly higher compared to $\mu = 0.2$ (left), suggesting that greater distribution shifts lead to higher initial errors. This behavior is observed for both step sizes and can be seen more precisely for all the mean shift (μ) values in Figure 6.4.

From Figure 6.3, we also observe that irrespective of the choice of the step size (γ) and mean shift (μ), the MSE dips below the OLS baseline before eventually converging towards it, implying that iterative optimization can momentarily outperform OLS but ultimately stabilizes around a similar error level. However, the convergence rate depends on the step size (γ), with higher γ helps in converging faster. This can be observed in Figure 6.3a, where the gradient descent with $\gamma = 0.1$ converges to the OLS baseline in ~ 40 iterations, whereas it takes ~ 80 iterations with $\gamma = 0.05$ (Figure 6.3b).

Overall, the results from Figure 6.3 highlight the impact of covariate shift on learning dynamics, where larger shifts exacerbate the initial error but do not necessarily affect the final convergence behavior. Also, the choice of step size (γ) affects the convergence rate to a great extent. Since our main goal is to minimize the MSE as much as possible, so if we can select an *optimal stopping time* (T_{opt}), then gradient descent estimator will give us a better result than the baseline of ordinary least squares estimator.

6.2. Effect of Mean Shift on MSE

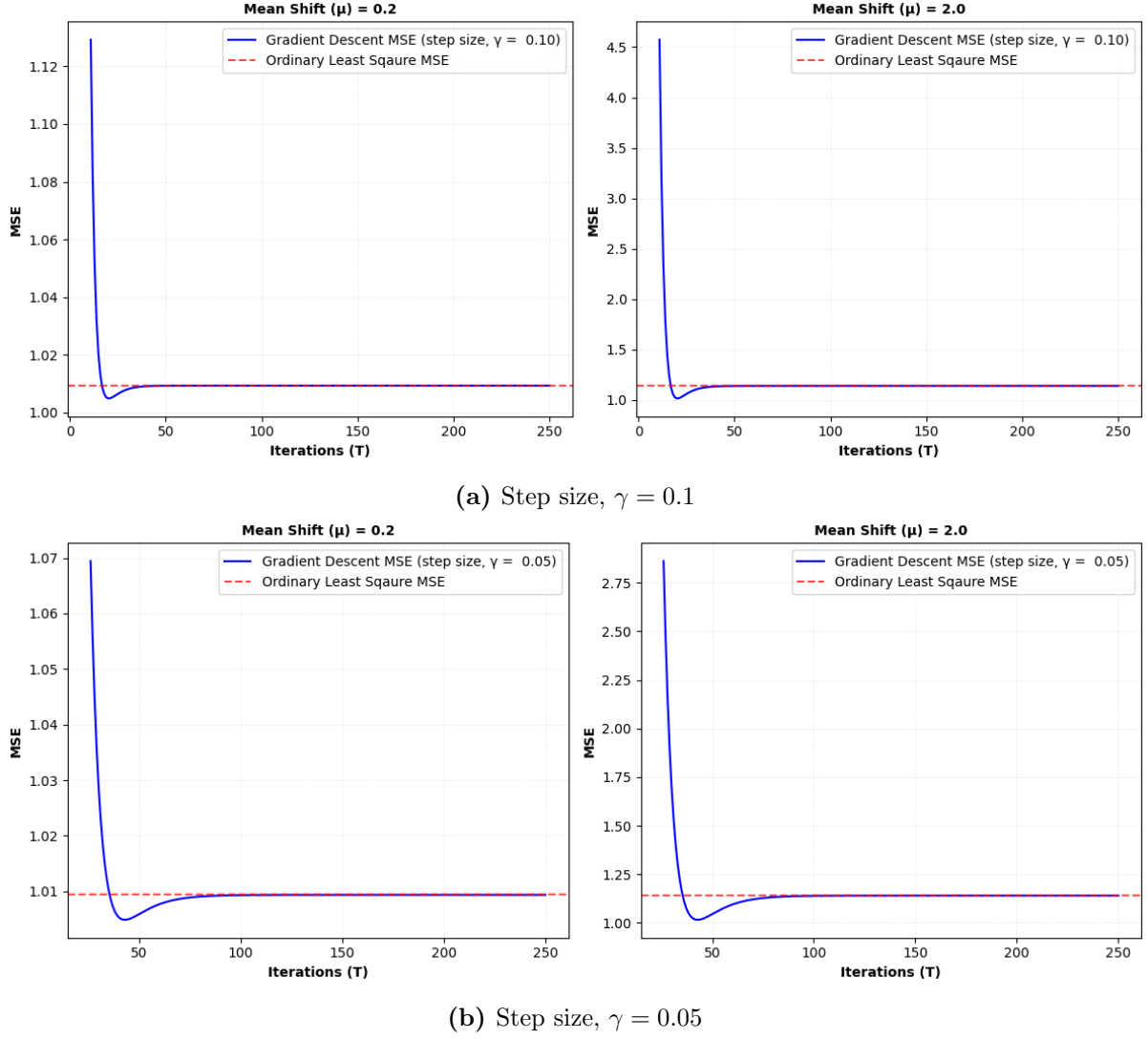


Figure 6.3: The image on left side plots the MSE against the increasing iterations for a mean shift, $\mu = 0.2$. The image on the right shows the same experiment for $\mu = 2.0$. We have chosen two extreme mean shift values to show the extreme effects. The experiment was carried out for two different step size, (a) $\gamma = 0.1$ and (b) $\gamma = 0.05$, to also observe the effect of step size (γ) on the convergence rate. **Note:** For better visualization purpose, the number of iterations on X-axis are truncated (initial few iterations are removed) in all plots to show the dip in curve prominently.

By theory, the bias in expected excess risk decreases with the number of iterations, while the variance in expected excess risk increases. At optimal stopping time (T_{opt}), we achieve a balance between these two (both becomes equal) and we found an upper bound of expected excess risk.

Proposition 6.2.2 (Optimal Stopping Time and Upper Bound of Expected Excess Risk). Choosing

$$T_{\text{opt}} = \frac{\sqrt{n} \|\theta^*\|}{\gamma \sigma \sqrt{2 \text{Trace}[\hat{\Sigma}]}} \quad (6.9)$$

the expected excess risk satisfies

$$\mathbb{E}[\mathcal{R}(\hat{\theta}_{\text{T}_{\text{opt}}})] - \mathcal{R}^* \leq \frac{4\sigma\|\theta^*\| \text{Trace}[\hat{\Sigma}]^{\frac{1}{2}}}{\sqrt{n}} \quad (6.10)$$

where n is the sample size, σ is the standard deviation in the noise ϵ and $\hat{\Sigma} = \frac{1}{n}\mathbf{X}^\top\mathbf{X}$ is the covariance matrix.

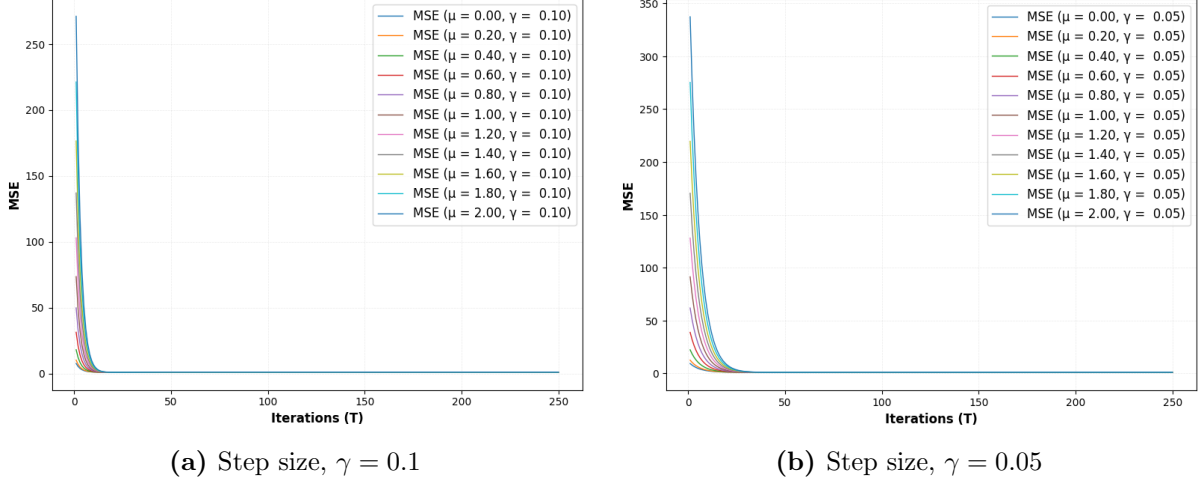


Figure 6.4: Initial value and rapid change in MSE with increasing iterations for multiple mean shifts, μ . With a higher μ , the initial value of MSE is greater compared to that with lower μ . Also, with (a) higher step size ($\gamma = 0.1$), gradient descent converges much faster than gradient descent with (b) lower step size ($\gamma = 0.05$).

Figure 6.5 shows, more prominently using a heatmap, what we already observed before. A lower value of μ results in a much greater MSE initially as compared to the MSE for a higher μ . As one can expect, for higher mean shift (μ), it also takes longer iterations for the gradient descent to converge and reach minimum. The convergence rate also depends on the step size (γ): a higher step size ($\gamma = 0.1$) (Figure 6.5a) achieves faster convergence than a lower step size ($\gamma = 0.05$) (Figure 6.5b).

The 3D plot in Figure 6.6 again visualizes the relationship between MSE (Mean Squared Error), iteration count in gradient descent (T), and mean shift (μ) using two different step sizes, $\gamma = 0.1$ and $\gamma = 0.05$. Both plot shows that for small values of μ , the MSE remains relatively low and stable as iterations increase. However, for larger values of μ , the initial MSE is significantly higher, indicating a stronger negative impact of distribution shift on model performance. As iterations progress, the MSE decreases and stabilizes, suggesting convergence. The steep rise at the beginning for large μ values highlights the sensitivity of the model to distributional changes.

Figure 6.7 shows how the gradient norm $\|\nabla\hat{\mathcal{R}}(\theta)\|$ (Figure 6.7a and Figure 6.7b) and parameter norm $\|\hat{\theta}^{\text{GD}}\|$ (Figure 6.7c and Figure 6.7d) change during the gradient descent iterations. Since we learn the parameter only from the training sample $\mathcal{S} \sim \mathcal{N}(0, 1)$, this does not depend on the mean shift (μ).

As we observe in Figure 6.7a and 6.7b, $\|\nabla\hat{\mathcal{R}}(\theta)\|$ initially starts with very high value and then with increasing iterations it steeply reduces, finally converging to 0 in ~ 30

6.2. Effect of Mean Shift on MSE

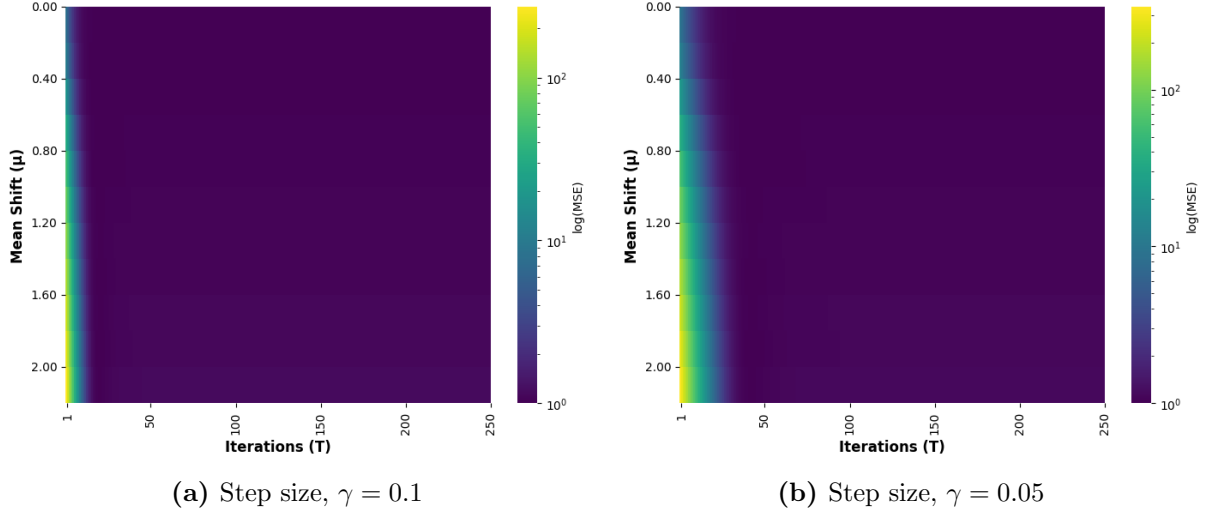


Figure 6.5: Heatmap of MSE to compare mean shift (μ), iterations (T) and MSE in a single plot. Higher mean shift results in greater MSE initially and naturally it takes longer iterations for the gradient descent to minimize the MSE. Results are illustrated with (a) $\gamma = 0.1$ and (b) $\gamma = 0.05$. The logarithm of MSE is taken in the color bar for better scaling.

iterations for $\gamma = 0.1$ and in ~ 60 iterations for $\gamma = 0.05$. This is because the gradient norm measures how large the gradient is at a given point. A large gradient norm means the function is steep, and updates will be large, whereas, a small gradient norm means the function is flat, and updates will be small. At the beginning, the gradient norm is high because the parameters are far from the optimal values. As iterations progress, the optimizer moves downhill in the loss landscape, reducing both the loss and the gradient norm. Closer to the minimum, the gradient norm approaches zero because the function is nearly flat (stationary point).

The same logic explains the observations in Figure 6.7c and 6.7d. Initially, $\|\hat{\theta}^{\text{GD}}\|$, representing the norm (magnitude) of the parameter vector θ , starts near zero. It rapidly increases within the first few iterations. The rate of increase slows down, and finally it plateaus at a final value around ≈ 3.2 in ~ 30 and ~ 60 iterations respectively for $\gamma = 0.1$ and $\gamma = 0.05$. We recall that the parameters are updated in iterations by taking a step towards the gradient direction: $\hat{\theta}_{t+1} = \hat{\theta}_t - \gamma \nabla \mathcal{R}(\hat{\theta}_t)$. At the beginning, gradient updates are large because the model parameters are far from optimal. As the optimizer updates θ through iterations, it moves towards an optimal region, causing the norm to increase. Eventually, closer to the minimum, the gradient approaches zero, meaning the parameters stabilize, and further updates are minimal.

Moreover, we also observe how it takes significantly less number of iterations for both the norms to reach its optimal value by increasing the step size from $\gamma = 0.05$ to $\gamma = 0.1$.

6.2.3 Comparison between OLS and GD estimator (Effect of Mean Shift)

We finally compare the performance from both ordinary least squares and gradient descent estimator (with different step sizes) for varying μ to observe the affect of mean shift on their performance.

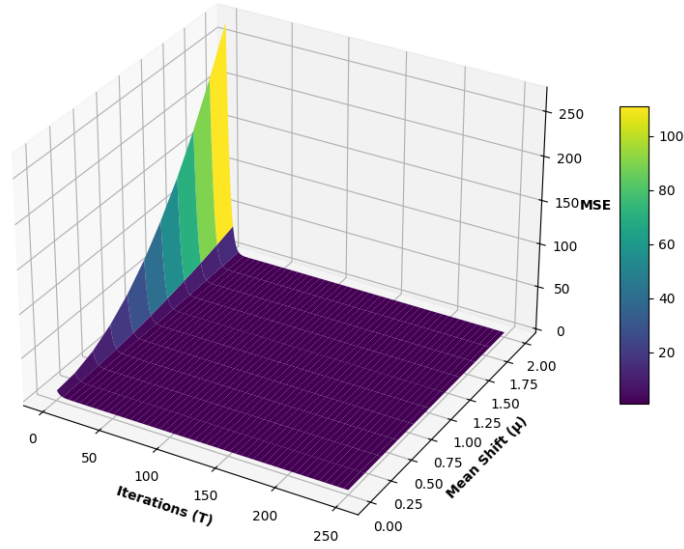
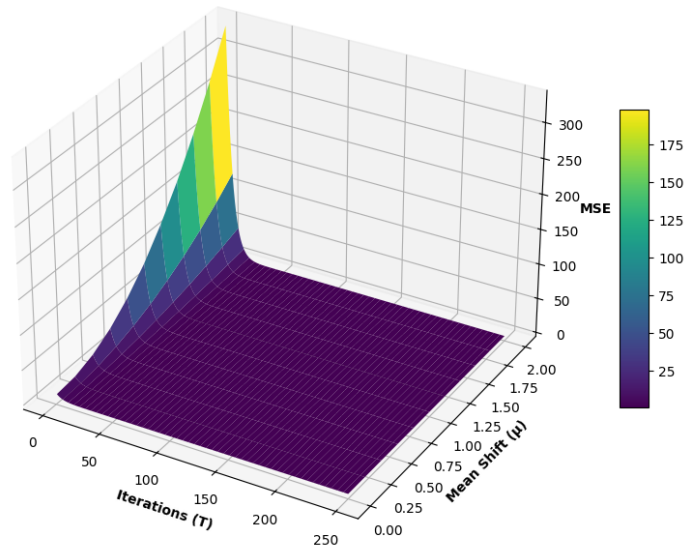
(a) Step size, $\gamma = 0.1$ (b) Step size, $\gamma = 0.05$

Figure 6.6: 3-dimensional surface plot of interplay among mean shift (μ), iteration count (T) and the respective mean square error (MSE) value in gradient descent method. The experiment was run using both (a) step size, $\gamma = 0.1$ and (b) step size, $\gamma = 0.05$.

The three-panel plot in Figure 6.8 plots the Mean Squared Error (MSE) obtained in Gradient Descent (GD) and Ordinary Least Squares (OLS) for different mean shift (μ). We have plotted the result for three different iteration count (T): the leftmost plot corresponds to $T = 25$, the middle plot to $T = 75$, and the rightmost plot to $T = 250$. The three plots above (Figure 6.8a) represents gradient descent with step size, $\gamma = 0.1$ while the three plots below (Figure 6.8b) represents gradient descent with step size, $\gamma = 0.05$.

6.3. Regularizing Covariate Shift

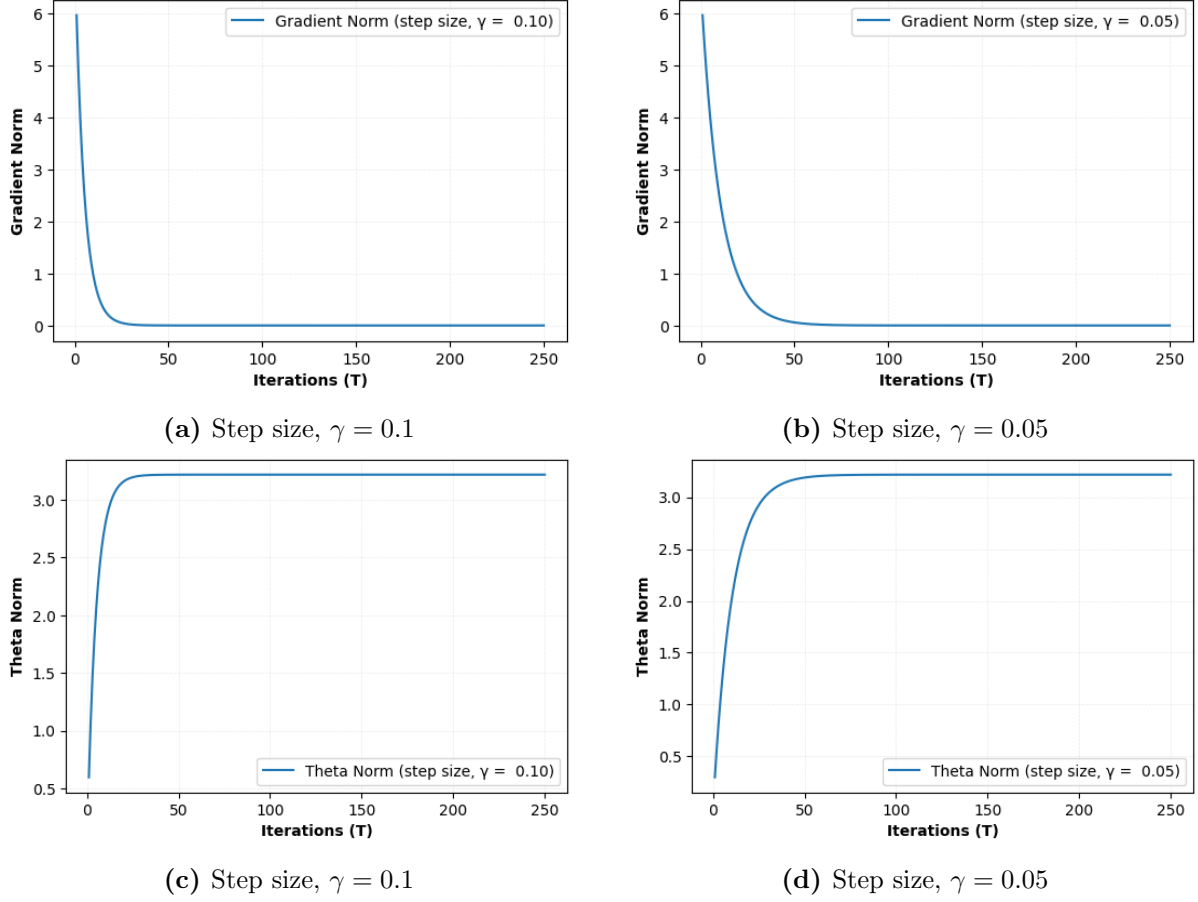


Figure 6.7: Illustration of how (a), (b) the gradient norm $\|\nabla \hat{\mathcal{R}}(\theta)\|$ and (c), (d) parameter norm $\|\hat{\theta}^{\text{GD}}\|$ change across iterations during the training using gradient descent method.

We can observe that initially at $T = 25$, the GD MSE (blue) is significantly higher than the OLS MSE (green) for both values of γ , indicating that GD has not fully converged. Moreover, lower step size produces a smoother curve as the updates in each iterations are slower. As iterations increase to $T = 75$, the GD MSE with $\gamma = 0.05$ aligns almost closely with the OLS MSE, while the GD MSE with $\gamma = 0.1$ has already converged with the OLS MSE, suggesting better convergence. At $T = 250$, GD and OLS produce identical errors, confirming that GD has converged to the optimal solution.

Additionally, across all plots, the MSE increases with μ for both gradient descent and OLS, indicating that a larger mean shift negatively impacts predictive accuracy. This behavior is expected since shifting the mean introduces distributional changes, making model estimation less effective. Overall, the results highlight that GD requires sufficient iterations to achieve optimal performance but can eventually approximate OLS solutions when properly tuned. The number of iterations also depends on the step size, γ .

6.3 Regularizing Covariate Shift

In the following section, we will try to regularize the affect mean shift (μ) using importance weighting. We will use two different methods to estimate the weights: *kernel density*

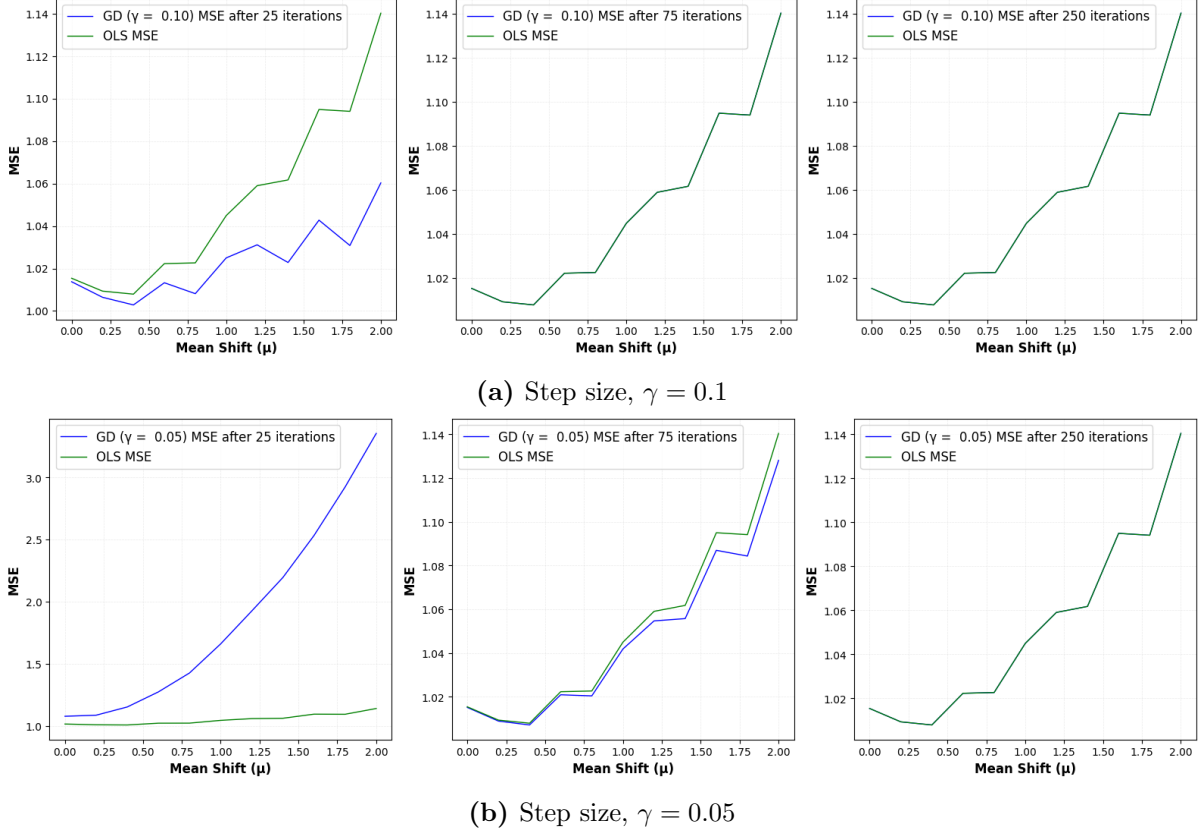


Figure 6.8: Mean Squared Error (MSE) of Gradient Descent (GD) and Ordinary Least Squares (OLS) as a function of Mean Shift (μ), for different numbers of iterations (T): (I) results at T = 25, (II) results at T = 75 and (III) results at T = 250. This is to observe the results during the beginning, after a few iterations and at the very end of the total iterations in gradient descent. The experiment is done twice for different step sizes: (a) $\gamma = 0.1$ and (b) $\gamma = 0.05$

estimation and self-attention mechanism.

6.3.1 Importance Weighting using Kernel Density Estimation

As we recall from section 3.1, if $p_{\text{train}}(X)$ and $p_{\text{test}}(X)$ are the probability density functions of the input distribution $\mathbb{P}_{\text{train}}(X)$ and $\mathbb{P}_{\text{test}}(X)$ respectively, then the *importance weights* can be estimated as the ratio of test and training input densities at training input points:

$$\omega(x_i) = \frac{p_{\text{test}}(x_i)}{p_{\text{train}}(x_i)} \quad (6.11)$$

$\mathbb{P}_{\text{train}}(X)$ and $\mathbb{P}_{\text{test}}(X)$ are also called *source distribution* and *target distribution* respectively.

In practical situations, we do not know which distribution is the training and test samples are drawn from. So the underlying probability density functions are also unknown. In such cases, we try to estimate the probability density function (PDF). One such method is *Kernel Density Estimation*.

Importance Weighted variant: Ordinary Least Square (OLS)

We use the closed form solution in equation 3.10 to compute the MSE in importance weighted setting using OLS estimator. For our experimental purpose, we have used *Gaussian Kernel* to estimate the densities, as mentioned in equation 3.5.

Figure 6.9 compares the MSE calculated through standard OLS estimator (unweighted) and its importance weighted variant across varying mean shift (μ). The comparison is done at different bandwidth level.

We recall that bandwidth controls the smoothness of the kernel density estimate. A smaller bandwidth produces a more detailed (possibly noisy) density estimate while a larger bandwidth produces a smoother density estimate, which might oversmooth the data and lose details. This behavior can be observed in Figure 6.9. While with a smaller bandwidth, the estimated density is noisy (irregular), making the difference between MSE calculated for unweighted and weighted OLS greater and erratic. With a higher bandwidth ($h > 2.0$), the density becomes smoother and stable, hence the MSE of weighted OLS approaches towards the MSE of unweighted OLS. There might be even some loss of details contributing to this convergence.

We also observe in Figure 6.9 that the difference between MSE calculated for unweighted and weighted OLS increases with higher mean shift (μ). This behavior is expected since shifting the mean introduces distributional changes, which results in the greater difference in respective MSE values.

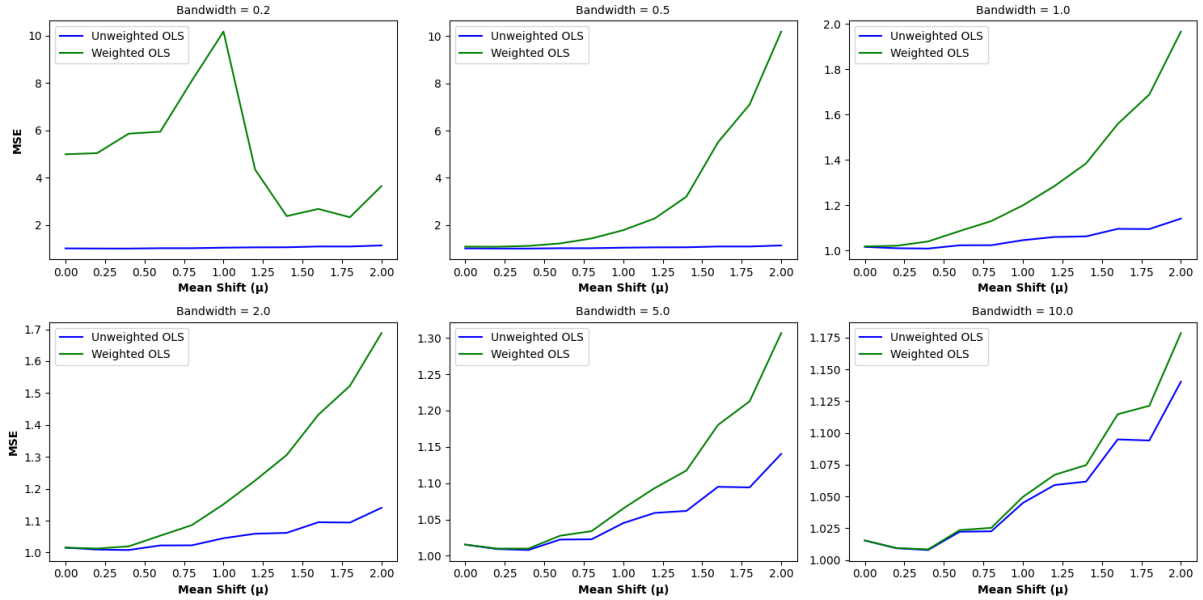


Figure 6.9: Effect of Importance Weighting (Kernel Density Estimation) with different bandwidth on MSE in Mean Shift setup. The MSE is calculated for both standard ordinary least square (unweighted) and importance weighted least square estimator at different bandwidth: (a) $h = 0.2$, (b) $h = 0.5$, (c) $h = 1.0$, (d) $h = 2.0$, (e) $h = 5.0$, (f) $h = 10.0$.

But even with the importance weighting, the regularization effect is not prominent since

the MSE for weighted variant of OLS is consistently higher than that of unweighted variant (Figure 6.9). This can have multiple reasons. To investigate further whether the sample size plays any role in the effect of mean shift on MSE, we plot the MSE for different sample sizes. At this moment, we **fix the mean shift** at $\mu = 1.0$ and use **fixed bandwidth** = 0.5 (using *Silverman's rule of thumb*: $h = 1.06 \times \sigma \times n^{-\frac{1}{5}}$, where σ is the standard deviation of the data and n is the sample size).

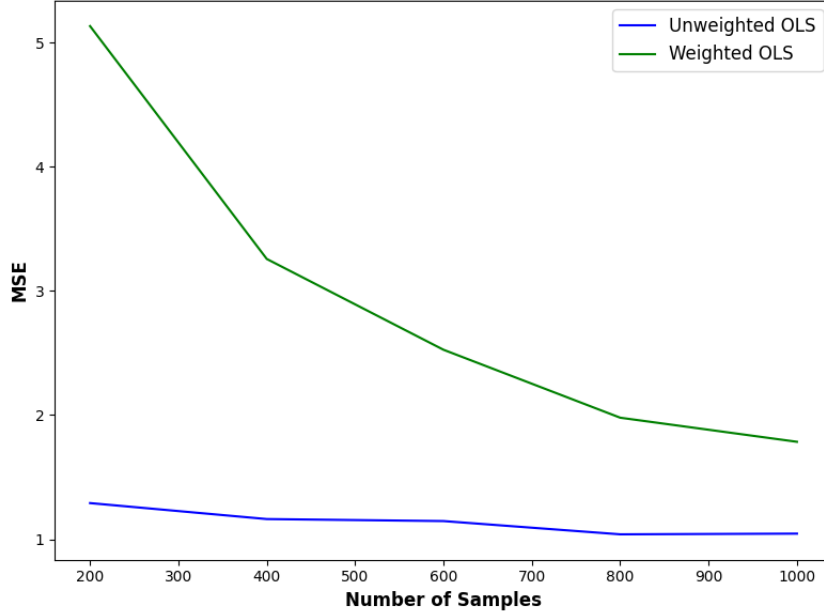


Figure 6.10: Effect of sample size on mean shift regularization using importance weighting. The comparison is done between unweighted ordinary least square and importance weighted ordinary least square method at fixed mean shift ($\mu = 1.0$) and fixed bandwidth ($h = 0.5$).

As we observe in Figure 6.10, the sample size indeed plays a role. With higher sample sizes, the MSE calculated with weighted variant of OLS decreases rapidly and approaches the MSE of unweighted OLS. Due to limitation of computational resources, we couldn't experiment with any higher sample size, however this behavior is consistent with what we already discussed in section 4.3.2, equation 4.20:

$$\lim_{n \rightarrow \infty} (\hat{\theta}_{\text{IWOLS}}) = \theta^*$$

Further investigation is required to investigate whether any other factors other than sample size plays a crucial role in regularizing the effect of mean shift by importance weighting.

Importance Weighted variant: Gradient Descent (GD)

Similar to OLS, We use 3.11 and 3.12 to compute the MSE in importance weighted setting using OLS estimator. At this moment, we fix the mean shift at $\mu = 1.0$ and step size, $\gamma = 0.1$, bandwidth = 0.5 and increase the iteration count, T to 5000.

Figure 6.11 shows the effect of importance weighting in regularizing the effect of mean shift by comparing the MSE calculated with both unweighted and weighted variants of

6.3. Regularizing Covariate Shift

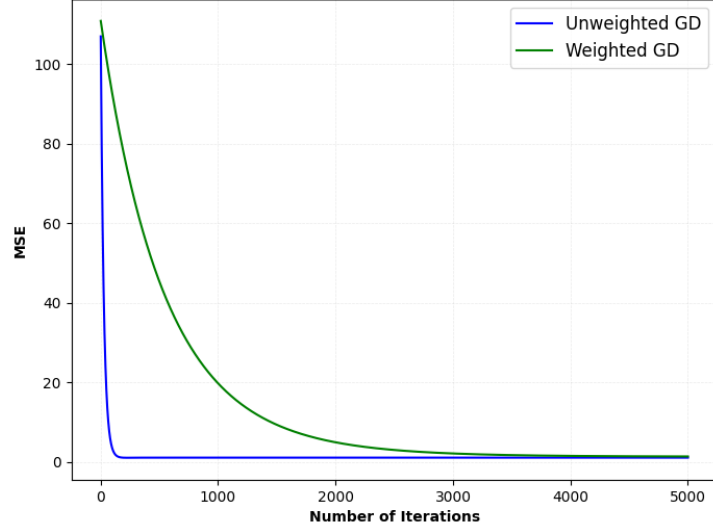


Figure 6.11: Effect of importance weighting in mean shift regularization: Comparing MSE with unweighted and weighted variants of gradient descent, computed with step size, $\gamma = 0.1$ and iteration count, $T = 5000$. The experiment is done with fixed mean shift, $\mu = 1.0$ and bandwidth $= 0.5$.

gradient descent method. We observe here that with increasing iterations, the MSE calculated with weighted version of gradient descent is reducing steadily and approaching towards the MSE calculated with unweighted version of gradient descent, finally converging in ~ 4000 iterations. It was observed during the experiment that step size (γ) plays a crucial role, reducing it will take significantly more number of iterations to converge.

Once again, we try to investigate whether sample size plays any role in regularizing effect. As we observe in Figure 6.12, increasing the sample size helps in regularizing the effect of mean shift. Keeping the number of iterations fixed at $T = 5000$, we can see that as sample size increases, performance of weighted gradient descent steadily approaches towards that of unweighted version and *almost* converges with 1000 sample sizes. Working with further sample size will require more computational resources.

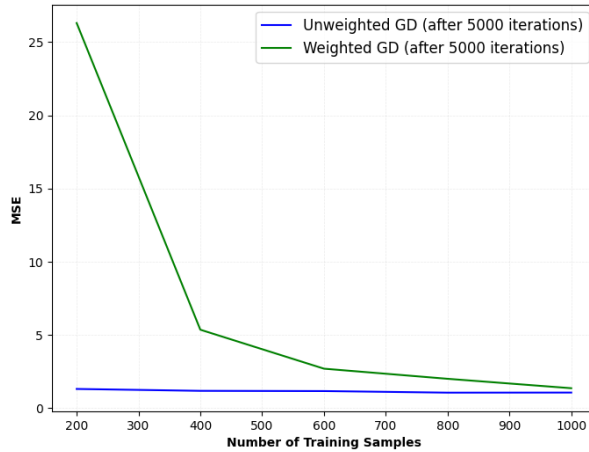


Figure 6.12: Effect of sample size on regularizing mean shift by importance weighted gradient descent with $T = 5000$ iterations, step size, $\gamma = 0.1$ and sample size $= 1000$.

In Figure 6.13, we compare the performance of weighted OLS and weighted GD estimator, with a step size, $\gamma = 0.1$. We can observe here that unlike in the unweighted setup (Figure 6.3), in the weighted setup, the gradient descent converges to OLS very steadily even with the same step size. It takes significantly higher number of iterations to converge to OLS compared to unweighted setup.

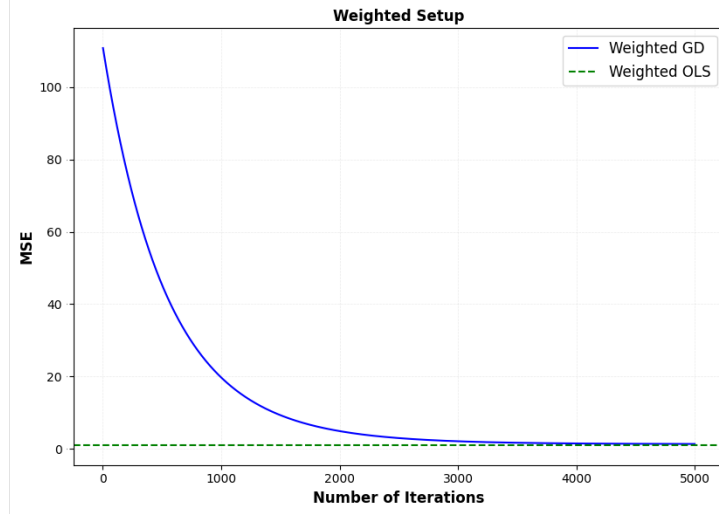


Figure 6.13: Comparison of weighted OLS and weighted GD (step size, $\gamma = 0.1$) performance across iterations for fixed mean shift, $\mu = 1.0$.

One of the reason for this erratic behavior is of course is the sample size, as we have already established. Figure 6.14 shows the effect of sample size on regularizing mean shift by importance weighted variants of both OLS and gradient descent. Here we again compare the performance of gradient descent with that of OLS, in both weighted and unweighted setup, but in terms of sample size while keeping the iteration count fixed at $T = 5000$. This is in contrast to Figure 6.13, where we had kept the sample size constant at $n = 1000$ and measured the performance against varying iteration count, T , in weighted setup.

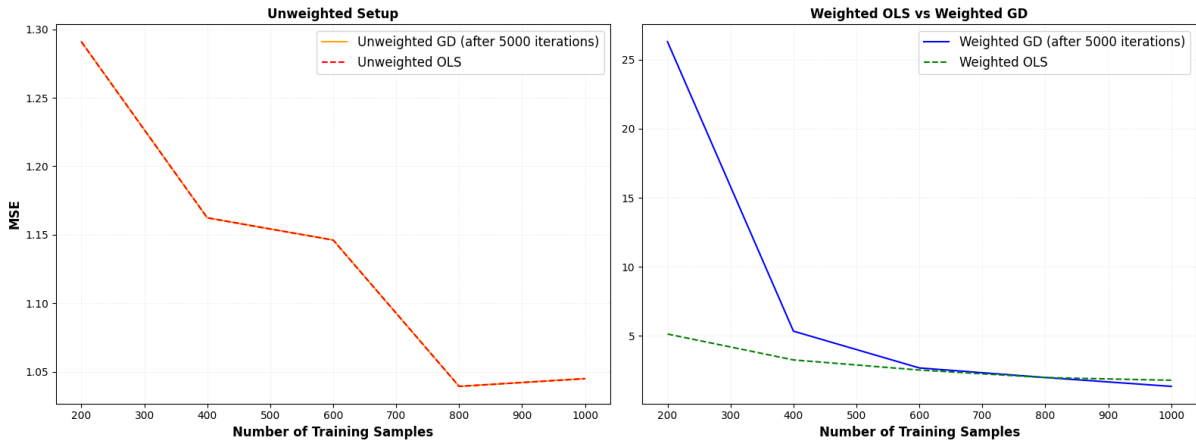


Figure 6.14: Effect of sample size on performance of OLS and gradient descent with step size, $\gamma = 0.1$ after $T = 5000$ iterations in both weighted and unweighted setup.

Our observation from Figure 6.14 solidifies our learning so far that sample size plays a

6.3. Regularizing Covariate Shift

crucial role in importance weighting. For same number of iterations, the performance of importance weighted gradient descent and convergence with importance weighted OLS depends on the sample size. In our current setup, with ~ 600 samples the weighted gradient descent could converge to weighted OLS performance and with even higher sample size (> 900), the weighted gradient descent could even beat the performance of weighted OLS.

We should also note that in the unweighted setup as well, the performance of gradient descent depends on the sample size. This can be observed in Figure 6.15. This is same as the previous figure, but the results are plotted after $T = 400$ iterations.

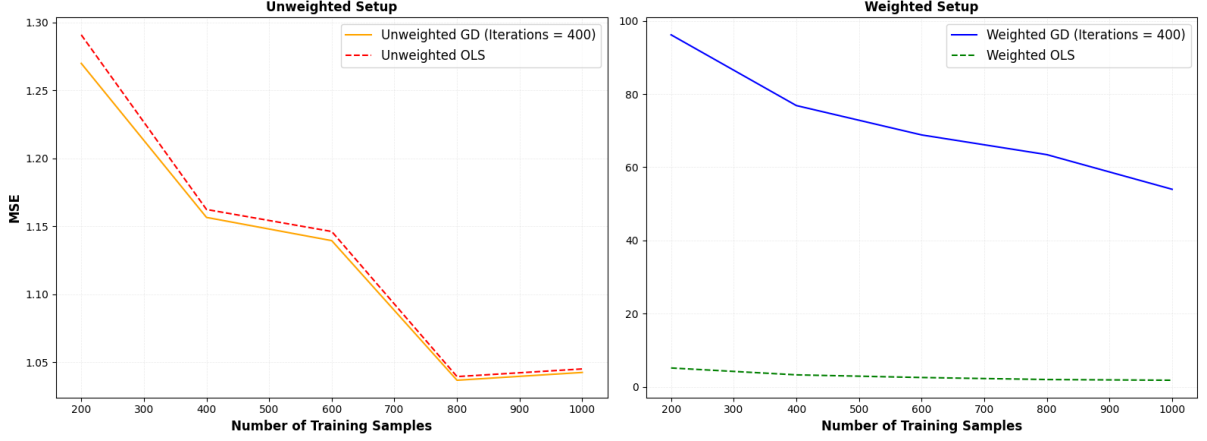


Figure 6.15: Effect of sample size on performance of OLS and gradient descent with step size, $\gamma = 0.1$ after $T = 400$ iterations in both weighted and unweighted setup.

We can observe that in unweighted setup, even with a relatively low iteration of $T = 400$, a fairly lower sample size is enough for the performance of gradient descent to converge with that of OLS estimator. With higher iterations, even lower sample size can be enough for the convergence.

6.3.2 Self-attention Mechanism

We recall from section 3.1.2 that for an empirical loss

$$\hat{\mathcal{R}}(\theta, \omega) := \frac{1}{n} \sum_{i=1}^n m(\omega^{(i)}) \ell(y_i, \langle \theta, x_i \rangle) \quad (6.12)$$

where $\omega = (\omega^{(1)}, \dots, \omega^{(n)})$ is the importance weights and $m : \mathbb{R} \rightarrow \mathbb{R}$ is a masking function, the *gradient descent-ascent algorithm* with initial regression parameter, $\theta_0 = 0$ and initial weights, $\omega_0^{(i)}$ for all $i = 1, \dots, n$ is given by:

$$\begin{aligned} \theta_{t+1} &= \theta_t - \gamma_t \nabla_{\theta} \hat{\mathcal{R}}(\theta_t, \omega_t) \\ \omega_{t+1} &= \omega_t - \eta_t \nabla_{\omega} \hat{\mathcal{R}}(\theta_t, \omega_t) \end{aligned} \quad (6.13)$$

where γ_t and η_t are the learning rates.

We compute the gradients of the above empirical risk with respect to ω and θ by:

$$\begin{aligned}
\nabla_{\theta} \hat{\mathcal{R}}(\theta, \omega) &= \frac{2}{n} \mathbf{X}^{\top} \mathbf{M}(\mathbf{X}\theta - \mathbf{Y}) \\
\nabla_{\omega} \hat{\mathcal{R}}(\theta, \omega) &= \frac{1}{n} \frac{dM(\omega)}{d\omega} (\mathbf{X}\theta - \mathbf{Y}) \odot (\mathbf{X}\theta - \mathbf{Y})
\end{aligned} \tag{6.14}$$

For our experimental purpose, we have kept the step size, $\gamma = 0.1$ and iteration count, $T = 7500$ for the gradient descent-ascent algorithm. The experiment was carried out at a fixed mean shift, $\mu = 1.0$.

Figure 6.16 shows the effect of masking function is regularizing mean shift. The comparison involves unweighted gradient descent (GD) and weighted GD with Self-Attention-based Masking, using different masking strategies. As we recall from the definition 3.1.1 of masking function, using $q = 0$ in the unbounded masking function $m_q(t) = t^q$ gives us unweighted variant, as $q = 0$ ignores the effect of weighting.

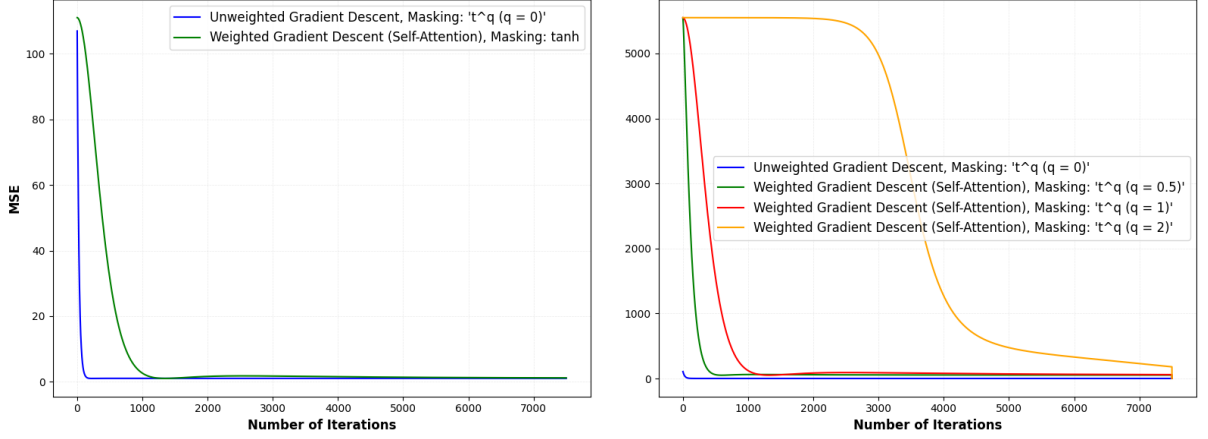


Figure 6.16: Regularizing mean shift by importance weighting using self-attention mechanism. The gradient descent-ascent algorithm was carried out at step size, $\gamma = 0.1$ and for 7500 iterations. Left: uses $m(t) = \tanh(t)$ masking function (bounded). Right: uses $m_q(t) = t^q$ masking function (unbounded) for $q \in [0.5, 1, 2]$.

The left subplot in Figure 6.16 uses a bounded masking function $m(t) = \tanh(t)$. Both unweighted and weighted gradient descent methods exhibit a sharp initial drop in MSE, indicating rapid convergence. After a certain number of iterations (~ 1000), both methods stabilize, reaching nearly the same final MSE. Also, the weighted GD initially decreases slower compared to unweighted GD but eventually reaches similar performance.

The right subplot in Figure 6.16 uses an unbounded masking function $m_q(t) = t^q$ and plots for different values of $q \in [0.5, 1, 2]$. We can observe that for smaller values of $q (= 0.5)$, weighted gradient descent converges comparatively faster. However, for larger values of q ($q = 2$), the initial MSE is extremely high, and convergence is much slower, possibly due to overemphasis on certain data points, leading to instability before gradually decreasing. This suggests that *larger q values in the self-attention weighting mechanism can lead to unstable learning process resulting in poor initial performance and slow adaptation*. This behavior aligns with our theoretical expectation from choosing high q value

6.3. Regularizing Covariate Shift

in unbounded masking function $m_q(t) = t^q$, which we discussed in section 3.1.2 (Choice of Masking Function).

Overall, we also observe from Figure 6.16 that the initial MSE for bounded masking function $m(t) = \tanh(t)$ is much lower than the unbounded masking function $m_q(t) = t^q$. It can also be noted that the unbounded masking function with $q = 1$ takes almost same number of iterations to converge to the unweighted gradient descent MSE as the bounded masking function.

We also want to see how sample size plays a role in the choice of masking function during the importance weighting with self-attention mechanism.

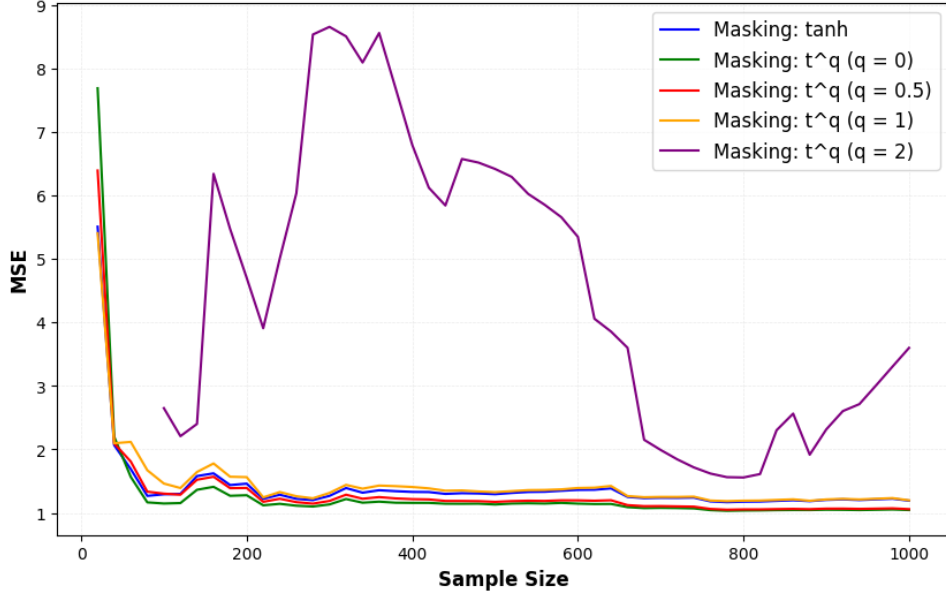


Figure 6.17: Effect of sample size on mean shift regularization with weighted gradient descent (step size, $\gamma = 0.1$ and number of iterations = 7500) using self-attention mechanism. Here, we use different masking functions, both unbounded and bounded.

In Figure 6.17, we observe that with same sample size ($n = 1000$), unbounded masking function with low value of q ($q = 0.5$) gives the best result for low sample sizes. We note here that the green line corresponding to $q = 0$ is the unweighted gradient descent here. For higher sample sizes (> 800), the performance of most masking functions are almost similar with very less difference. We can easily observe that the only exception is the unbounded masking function $m_q(t) = t^q$ with highest value of q ($q = 2$), because high value of q leads to unstable learning process. This aligns with our observation from Figure 6.16 and theoretical expectations discussed in section 3.1.2 (Choice of Masking Function).

7 Conclusion

Our experimental setup investigates the impact of covariate shift in linear regression models, particularly focusing on how a shift in the input mean affects Mean Squared Error (MSE) for different estimators. The study is conducted using both Ordinary Least Squares (OLS) and Gradient Descent (GD) estimators, followed by regularization techniques like importance weighting using Kernel Density Estimation (KDE) and a Self-Attention Mechanism.

The experiments confirm that covariate shift significantly impacts model performance, particularly for standard estimators like OLS. Gradient Descent adapts better to shift conditions, especially when combined with importance weighting techniques. Regularization methods, such as kernel-based importance weighting and self-attention mechanisms, effectively reduce MSE, making them valuable strategies for mitigating the negative effects of distributional shifts. It was also found that sample size and proper selection of masking function in self-attention mechanism also plays a crucial role in the rate of convergence in regularizing the effect of covariate shift.

These findings highlight the necessity of incorporating robust adaptation techniques in machine learning models deployed in real-world scenarios where covariate shift is prevalent.

Bibliography

- Aitken, A. C. (1936). On least squares and linear combinations of observations. *Proceedings of the Royal Society of Edinburgh*, 55(1):42–48.
- Bach, F. (2024). *Learning Theory from First Principles*. MIT Press.
- Baldi, P., Brunak, S., and Stolovitzky, G. A. (1998). *Bioinformatics: The Machine Learning Approach*. MIT Press, Cambridge.
- Belkin, M. and Niyogi, P. (2004). Semi-supervised learning on riemannian manifolds. *Machine Learning*, 56(1-3):209–239.
- Bickel and Scheffer, T. (2007). Dirichlet-enhanced spam filtering based on biased samples. In Schölkopf, B., Platt, J., and Hoffman, T., editors, *Advances in Neural Information Processing Systems 19*, Cambridge, MA. MIT Press.
- Bishop, C. M. (1995). *Neural Networks for Pattern Recognition*. Oxford University Press, Oxford, UK.
- Borgwardt, K. M., Gretton, A., Rasch, M. J., Kriegel, H.-P., Schölkopf, B., and Smola, A. J. (2006). Integrating structured biological data by kernel maximum mean discrepancy. *Bioinformatics*, 22(14):e49–e57.
- Bousquet, O., Chapelle, O., and Hein, M. (2004). Measure based regularization. In Thrun, S., Saul, L., and Schölkopf, B., editors, *Advances in Neural Information Processing Systems 16*, Cambridge, MA. MIT Press.
- Cauchy, A.-L. (1847). Méthode générale pour la résolution des systèmes d’équations simultanées. *Comptes Rendus de l’Académie des Sciences*, 25:536–538.
- Duda, R. O., Hart, P. E., and Stork, D. G. (2001). *Pattern Classification*. Wiley, New York, 2 edition.
- Fishman, G. S. (1996). *Monte Carlo: Concepts, Algorithms, and Applications*. Springer-Verlag, Berlin.
- Ganin, Y., Ustinova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F., Marchand, M., and Lempitsky, V. (2016). Domain-adversarial training of neural networks. In *Journal of Machine Learning Research*, volume 17, pages 1–35.
- Gogolashvili, D., Zecchin, M., Kanagawa, M., Kountouris, M., and Filippone, M. (2023). When is importance weighting correction needed for covariate shift adaptation? *arXiv preprint*, arXiv:2303.04020.

- Gretton, A., Borgwardt, K., Rasch, M., Schölkopf, B., and Smola, A. (2012). A kernel two-sample test. *Journal of Machine Learning Research*, 13:723–773.
- Gretton, A., Smola, A., Huang, J., Schmittfull, M., Borgwardt, K., and Schölkopf, B. (2009). Covariate shift by kernel mean matching. In Quiñero-Candela, J., Sugiyama, M., Schwaighofer, A., and Lawrence, N. D., editors, *Dataset Shift in Machine Learning*, chapter 3, pages 131–160. MIT Press.
- Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, 2nd edition.
- Heckman, J. J. (1979). Sample selection bias as a specification error. *Econometrica*, 47(1):153–162.
- Hein, M. (2006). Uniform convergence of adaptive graph-based regularization. In *Proceedings of the 19th Annual Conference on Learning Theory (COLT)*, pages 50–64.
- Huang, J., Smola, A., Gretton, A., Borgwardt, K. M., and Schölkopf, B. (2007). Correcting sample selection bias by unlabeled data. In Schölkopf, B., Platt, J., and Hoffman, T., editors, *Advances in Neural Information Processing Systems 19*, pages 601–608, Cambridge, MA. MIT Press.
- Kanamori, T., Hido, S., and Sugiyama, M. (2009). A least-squares approach to direct importance estimation. *Journal of Machine Learning Research*, 10:1391–1440.
- Kimeldorf, G. S. and Wahba, G. (1971). Some results on tchebycheffian spline functions. *Journal of Mathematical Analysis and Applications*, 33(1):82–95.
- Ma, C., Pathak, R., and Wainwright, M. J. (2022). Optimally tackling covariate shift in rkhs-based nonparametric regression. *arXiv preprint*, arXiv:2205.02986.
- Mohri, M., Rostamizadeh, A., and Talwalkar, A. (2018). *Foundations of Machine Learning*. MIT Press, Cambridge, MA, 2 edition.
- Qin, J. and Lawless, J. (1994). Empirical likelihood and general estimating equations. *The Annals of Statistics*, 22(1):300–325.
- Rahimian, H. and Mehrotra, S. (2019). Distributionally robust optimization: A review. *arXiv preprint arXiv:1908.05659*.
- Rao, C. R. (1945). Information and accuracy attainable in the estimation of statistical parameters. *Bulletin of the Calcutta Mathematical Society*, 37:81–91.
- Schölkopf, B. and Smola, A. J. (2002). *Learning with Kernels*. MIT Press, Cambridge, MA.
- Shelton, C. R. (2001). *Importance Sampling for Reinforcement Learning with Multiple Objectives*. Phd thesis, Massachusetts Institute of Technology.
- Shimodaira, H. (2000). Improving predictive inference under covariate shift by weighting the log-likelihood function. *Journal of Statistical Planning and Inference*, 90(2):227–244.

- Sugiyama, M., Krauledat, M., and Müller, K.-R. (2000). Covariate shift adaptation by importance weighted cross validation. *Journal of Machine Learning Research*, 1:1–48.
- Sugiyama, M., Nakajima, S., Kashima, H., von Büna, P., and Kawanabe, M. (2008). Direct importance estimation with model selection and its application to covariate shift adaptation. In *Advances in Neural Information Processing Systems (NIPS)*, volume 20, pages 1433–1440.
- Vapnik, V. (1998). *Statistical Learning Theory*. John Wiley & Sons, Chichester.
- Wolpaw, J. R., Birbaumer, N., McFarland, D. J., Pfurtscheller, G., and Vaughan, T. M. (2002). Brain-computer interfaces for communication and control. *Clinical Neurophysiology*, 113(6):767–791.
- Zhao, H., Li, Z., Wang, C., and Zhang, Y. (2021). Domain-adaptive decision trees for handling covariate shift. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(6):1845–1860.